

Introducción a la Estructura y Arquitectura de Computadores

1.1. Definición de Estructura del Computador y Arquitectura del Computador.

1.2. Estructura y funcionamiento de los computadores.

1.3. Modelos “como servicio” de computación en la nube.
Infraestructura como servicio (IaaS)

Organización

- **Introducción a la Estructura y Arquitectura de Computadores**
 - 1.1. Definición de Estructura del Computador y Arquitectura del Computador.
 - 1.2. Estructura y funcionamiento de los computadores.
 - 1.3. Modelos de computación en la nube “como servicio”. Infraestructura como servicio (IaaS)

Arquitectura, estructura, tecnología

Arquitectura de un computador:

- Interface programador, Prestaciones, consumo,...
- Describe “lo que sucede”

Estructura de un computador:

- Es una estructura lógica que da forma a su arquitectura.
- Describe “cómo sucede”

Arquitectura son los **atributos visibles al programador**, como, por ejemplo: Conjunto de instrucciones (ISA).
Número de bit usados para representar datos.

Rendimiento

Las funcionalidades de una arquitectura se pueden conseguir con diferentes organizaciones internas o estructuras. **Estructura es la manera en la cual los componentes se relacionan unos con otros**

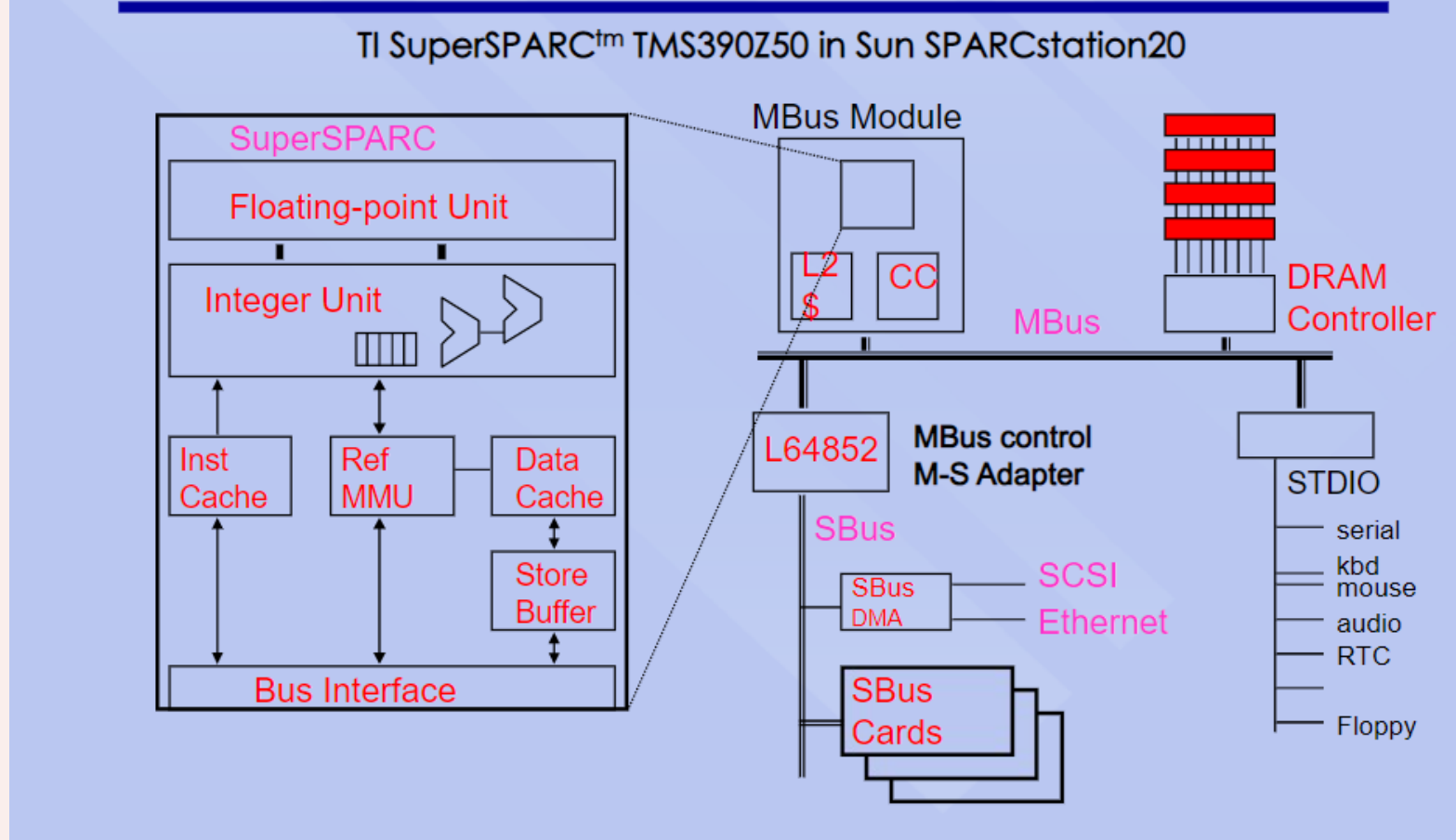
Conceptos importantes

- Estructura de un computador
 - Descomposición funcional
 - Fundamentos de la ejecución de un programa
- Evolución tecnológica
- Parámetros característicos (Tamaño de palabra, Frecuencia de reloj,...) y **evaluación del rendimiento**

La "organización o **estructura**" de un computador se refiere a la estructura lógica que da forma a su arquitectura

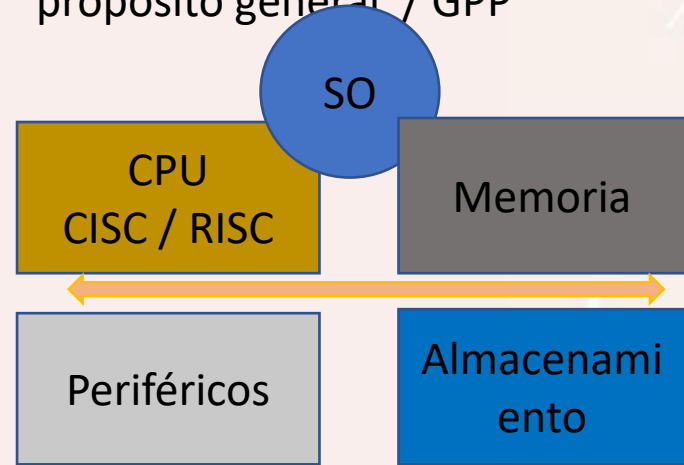
Estructura

Ejemplo de una Organización (década del 90)

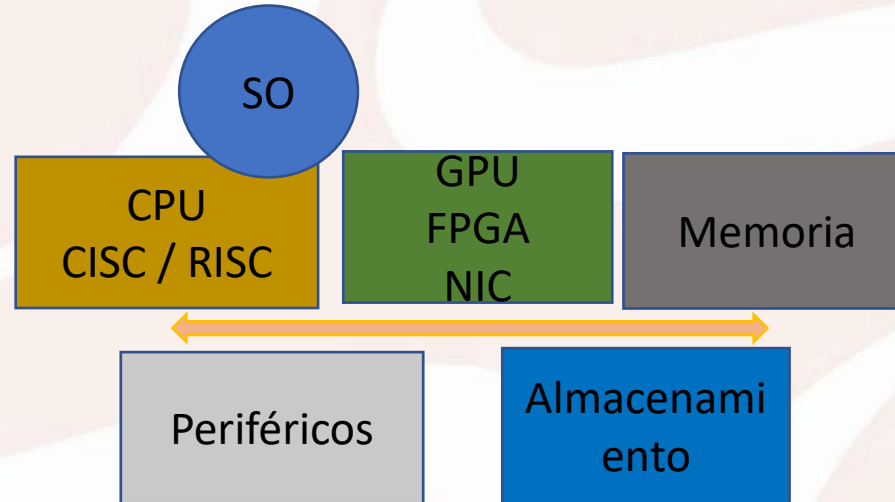


Sistema / Procesador

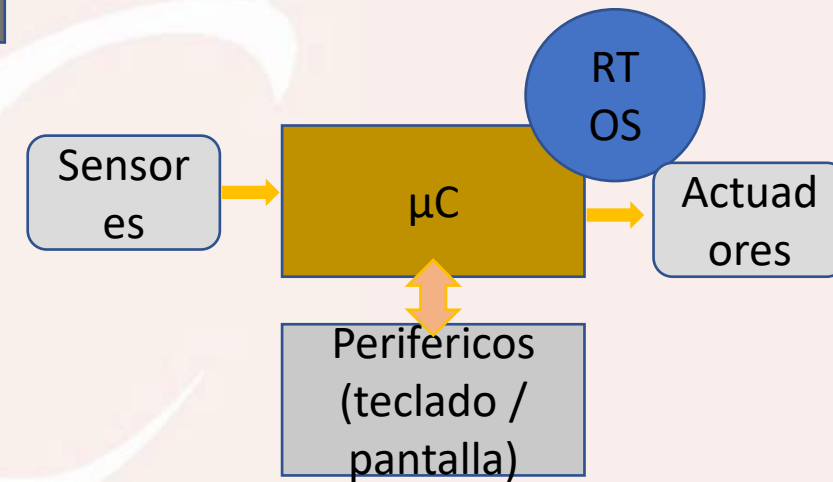
Sistema de procesamiento de propósito general / GPP



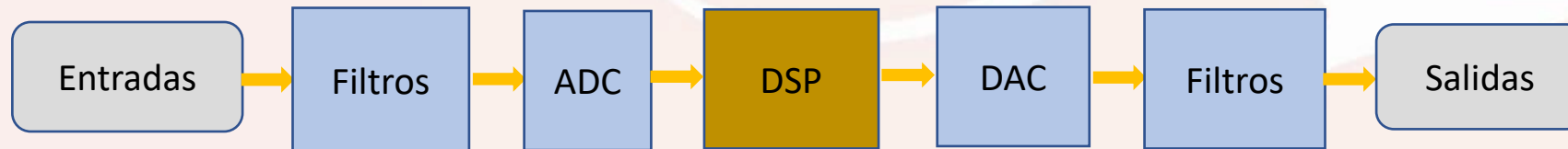
Sistema de con procesadores heterogéneos



Sistema de control (tiempo real) / μ C



Sistema de procesamiento de señal / DSP



Rendimiento del computador

Rendimiento de la CPU

- Características
 - ISA
 - Nº de unidades funcionales
 - Nº de registros,...
 - Nº de etapas
- Métricas
 - Tiempo de ejecución
 - IPS, OPS
- Estrategias
 - Paralelismo

Rendimiento de la memoria

- Características
 - Tamaño o capacidad número de palabras y bits por palabra (A mayor tamaño de memoria menor rapidez)
 - Entrelazado
- Métricas
 - Latencia o tiempo de respuesta en ns. Ligada a la rapidez de ejecución de los programas.
 - Ancho de banda
- Estrategia:
 - Jerarquía

Rendimiento de los buses

- Características
 - Número de líneas de comunicación.
 - Frecuencia de transmisión de los datos (Hz).
- Métricas
 - Ancho de banda (B/s).
 - Latencia
- Estrategia:
 - Organización: jerarquía

Rendimiento de los sistemas de cómputo

- Gestión adecuada de los recursos
- Dimensionamiento del sistema

Objetivo: reducir el tiempo de ejecución de una aplicación

¿Por qué no se ejecuta eficientemente?

¿Existen cuellos de botella



¿Dónde se encuentran los puntos críticos?

Regla 80/20:
parte significativa: los bucles

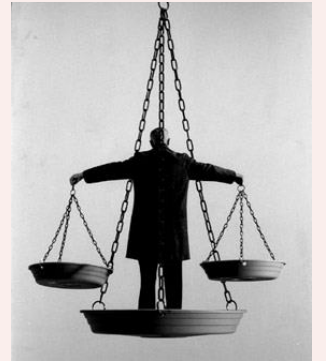
¿Cómo lograr el objetivo?

Optimización de cada uno de los componentes.
Paralelización
Organización: Jerarquía

Requisitos en el diseño de sistemas

- Nuevos enfoques : Velocidad / Coste / Disponibilidad / Usabilidad / Consumo
- Equilibrar entre tiempo / costo. Requiere
 - Conocimiento de la aplicación
 - Conocimientos de los tipos de recursos
 - Conocimientos operación / uso de los recursos
- Problema: encontrar el **equilibrio** tiempo / costo
 - Diseño
 - Producción
- Fundamentales otras consideraciones de diseño
 - Disponibilidad
 - Usabilidad

“ At LessHost.com, our **secure** data center & fast **tier-1** network provides the **reliability**, **performance**, & **scalability** required to maintain and **focus** on your core business, all while staying within your budget. ”



¿Por qué evaluar el rendimiento?

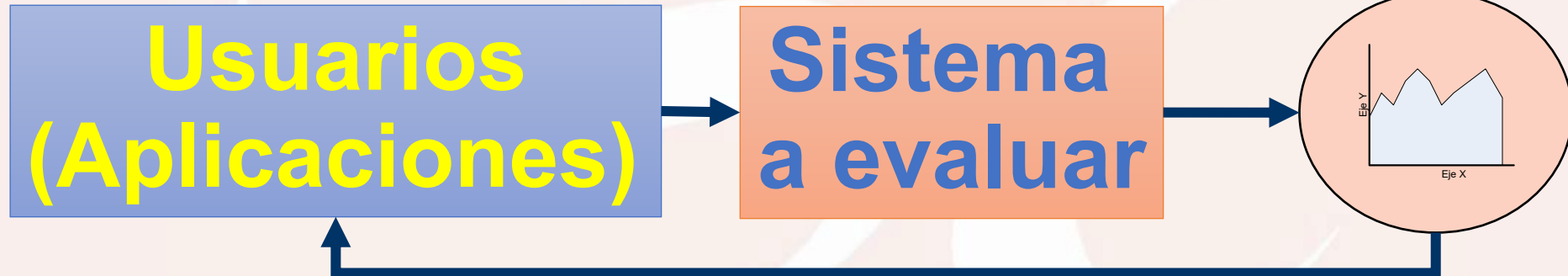
- ¿Qué interés tiene la evaluación?
- ¿Qué variables reflejan el rendimiento?
- ¿Qué afecta al rendimiento?

Evaluación del rendimiento : Carga de trabajo

Workload:
¿Qué aplicaciones?
¿Qué input?

Tiempo de ejecución
¿Cómo medir?

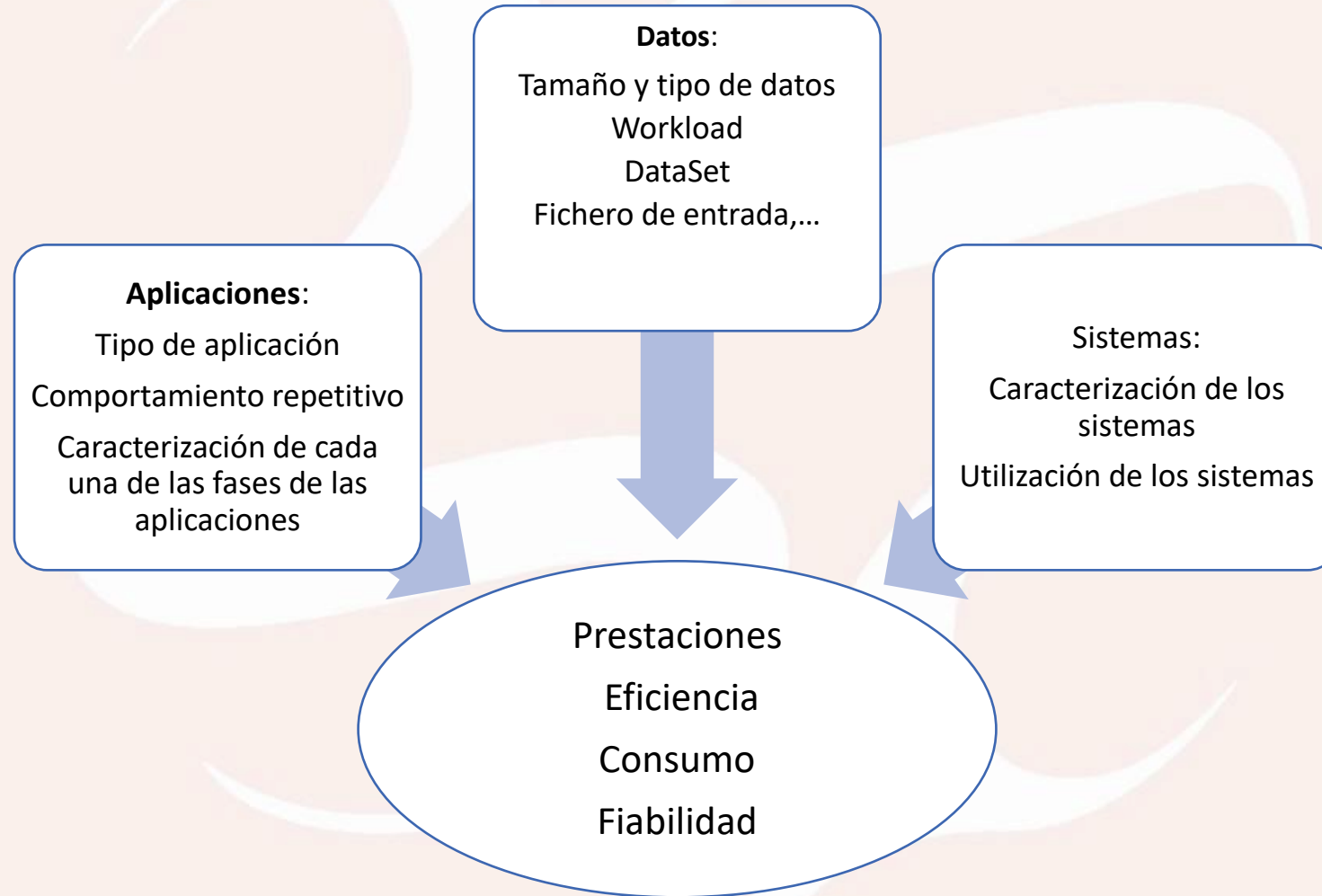
$$\text{Performance} = f(\text{Workload})$$



Para comparar diferentes sistemas hay que establecer un **criterio de medida**



Método para la evaluación de rendimiento del sistema de cómputo



“Iron Law” of Processor Performance (Ley de hierro)

- La ley de hierro es una ley o principio que es indiscutible e inevitable. Se puede **derivar mediante el uso de la lógica**, sin el conocimiento de ningún tema arduo.
- En la arquitectura y organización de computadoras, la **Ley de Hierro para calcular el tiempo de ejecución** viene dada por

$$Tej = NI \times CPI \times tc$$

Tej: Tiempo de ejecución

NI: (número de instrucciones ejecutadas)

CPI: (ciclos por instrucción)

tc: (tiempo de ciclo de reloj)

Rendimiento

- La medida más utilizada del rendimiento de un computador es la rapidez con la que puede ejecutar programas
- Para un buen rendimiento es necesario diseñar de modo coordinado:
 - Hardware
 - ISA
 - (Compilador)
- El tiempo T transcurrido para la ejecución depende de todas las unidades del computador

$$T_{eje} = NI * CPI / F$$

Factores que influyen en el rendimiento

- Rendimiento del procesador
 - NI: Nº real de instrucciones ejecutadas (No es necesariamente igual al nº de instrucciones del programa objeto)
 - CPI: Ciclos por Instrucción (media) (teniendo en cuenta Accesos a memoria)
 - Tc: Ciclo de reloj
 - F: Frecuencia (ciclos por segundo: 1/ciclo)

Si se mide el rendimiento como tiempo de ejecución

$$Numero_{ciclos_{programa}} = NI * CPI$$

$$T_{pejecución} = NI * CPI * Tc$$

$$T_{pejecución} = \frac{NI * CPI}{F}$$

$$Rendimiento = \frac{1}{T_{pejecución}}$$

Objetivo:

Aumentar rendimiento → reducir tiempo de ejecución

Algunas consideraciones sobre la tecnología

$$T_{\text{CPU}} = \text{NI} \times \text{CPI} / \text{frecuencia}$$

- Repertorio de instrucciones.
- Compilador
- Programador
- Algoritmo

Repertorio de instrucciones (ISA)
(RISC vs. CISC)
Microarquitectura

Tecnología (organización)
Diseño de circuitos
(en cada ciclo hay que hacer más cosas)

$$\text{tiempo}_{\text{ejecución}} = \frac{\text{Instrucciones}}{\text{programa}} \times \frac{\text{ciclos promedio}}{\text{instrucción}} \times \frac{\text{tiempo}}{\text{ciclo de reloj}} = \frac{\text{tiempo}}{\text{por programa}}$$

Ejercicio

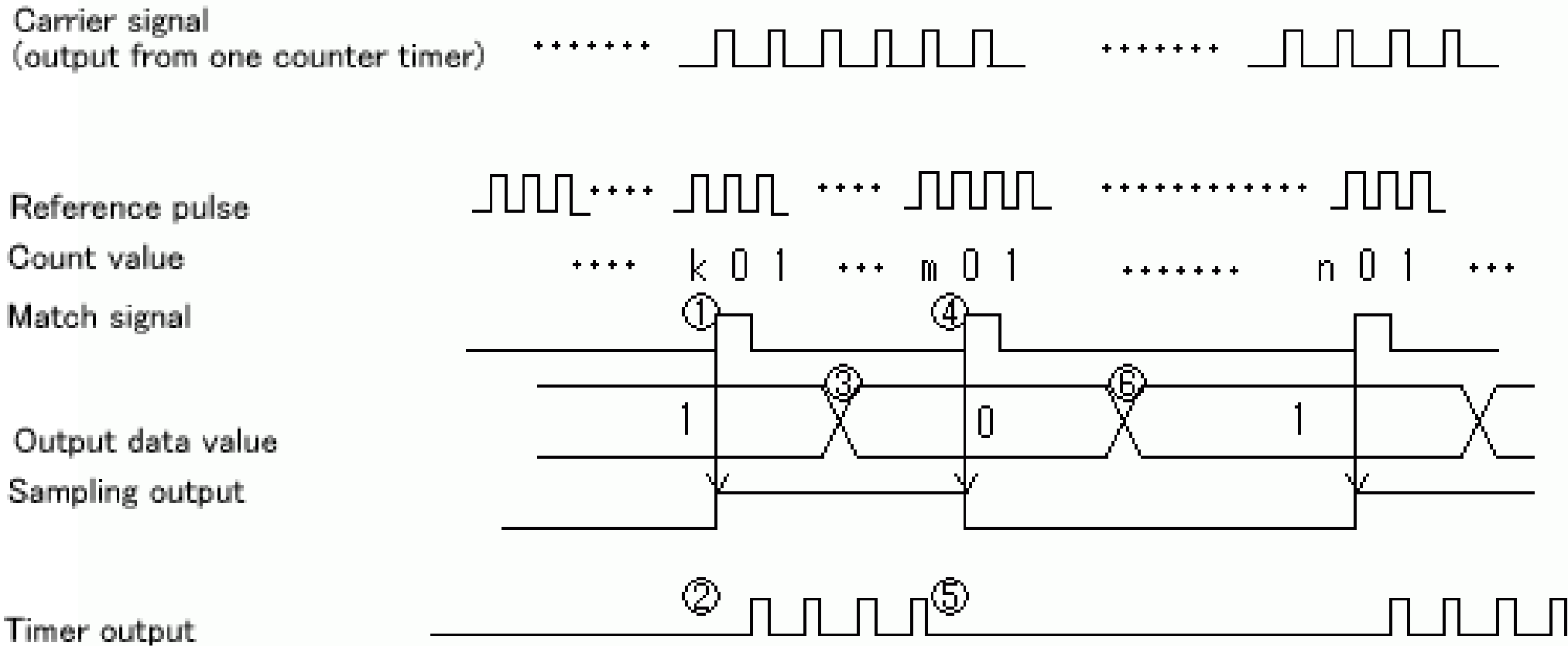
- Calcular el tiempo de CPU de un programa que necesita 100.000 ciclos si el reloj tiene una frecuencia de 500 MHz

- $T_{CPU} = \frac{1 \times 10^5}{500 \times 10^6} = 2 \times 10^{-4} = 20ms$

Clocks

- Cada computador tiene al menos un reloj que sincroniza las actividades de sus componentes.
- Para llevar a cabo cada operación o movimiento de datos es necesario un número fijo de ciclos de reloj
- La frecuencia del reloj, medida en MHz o GHz, determina la velocidad con la que se realizan las operaciones
- El tiempo de ciclo del reloj es el inverso de la frecuencia: $T=1/f$
 - Un reloj de 800 MHz tiene un tiempo de ciclo de 1.25ns.
 - $T=1/800 \text{ MHz}=1,25\text{ns}$

Señales: Clock – pulso – nivel



Carrier generation operation example

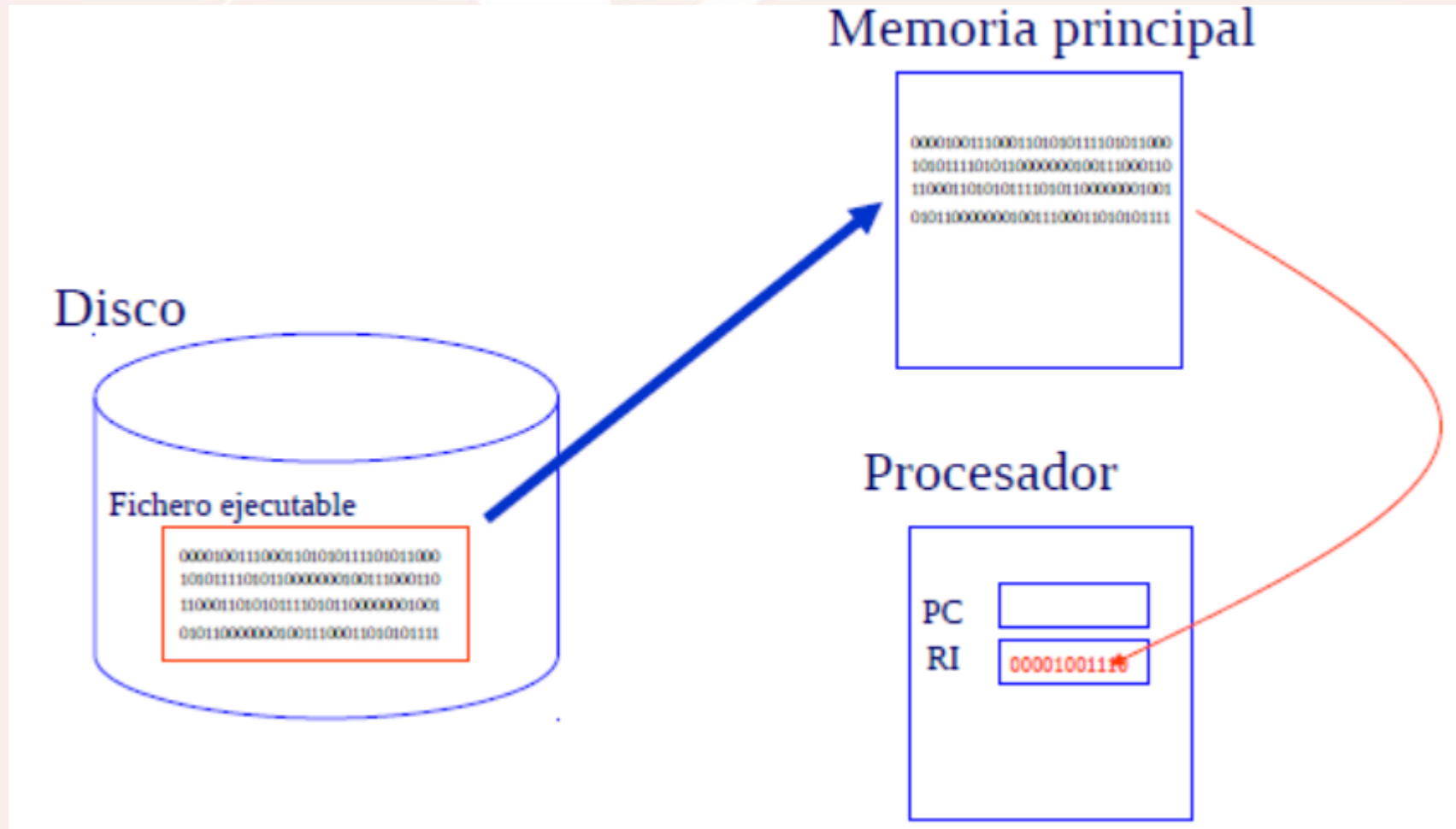
Clocks

- La **velocidad del reloj** no debe confundirse con las prestaciones.
- El tiempo de CPU requerido para ejecutar un programa depende de la ecuación general de las prestaciones:

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

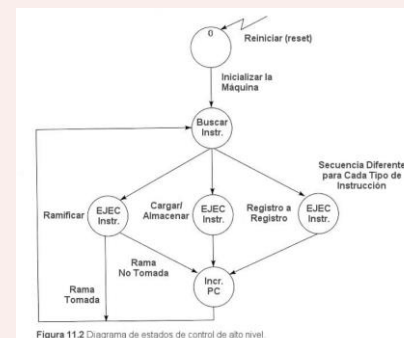
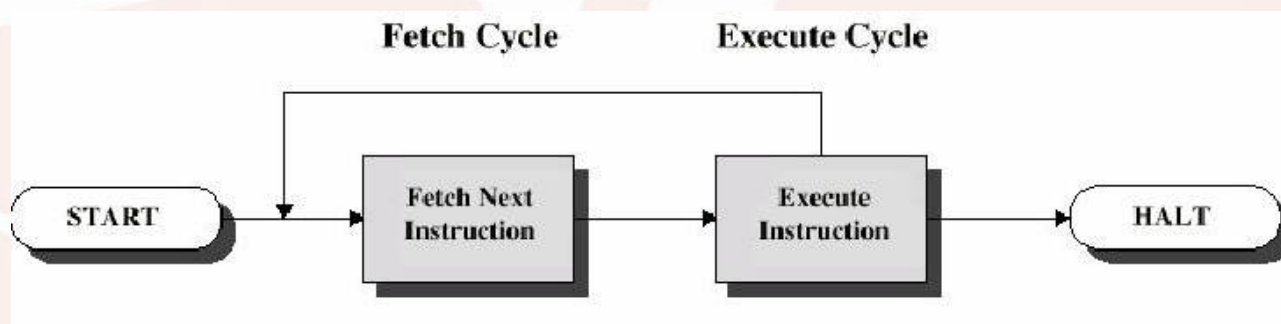
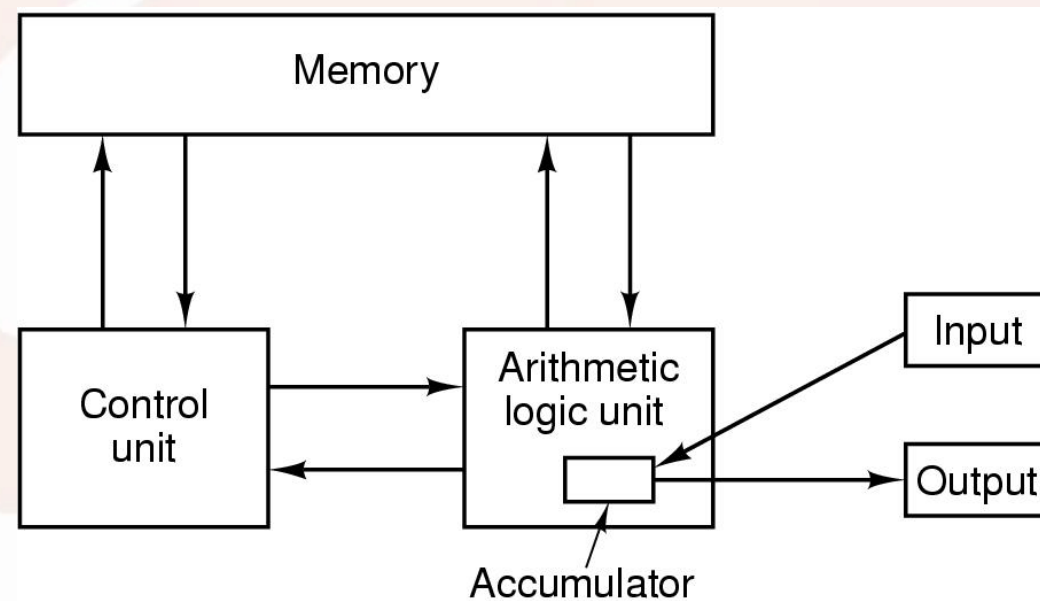
- Vemos que podemos mejorarlo cuando reducimos
 - El número de instrucciones en un programa
 - El número de ciclos por instrucción
 - El número de nanosegundos por ciclo de reloj

Funcionamiento: Carga y ejecución de un programa



Estructura de una máquina Von Neumann

- Cinco partes básicas
 - la memoria (M)
 - la unidad aritmética lógica (ALU)
 - la unidad de control (UC)
 - el equipo de entrada y salida (I/O)
 - Unidades de interconexión
- **Modelo de ejecución: FSM: Ciclo de instrucción**
 - Ciclo de búsqueda
 - Ciclo de decodificación
 - Ciclo de ejecución



Arquitectura de von Neumann

- John von Neumann propuso una manera de estructurar un computador.

- Almacenar

- Datos (leer - escribir).
 - Almacenar Programas.

} Memoria
de programa y datos

- Ejecutar

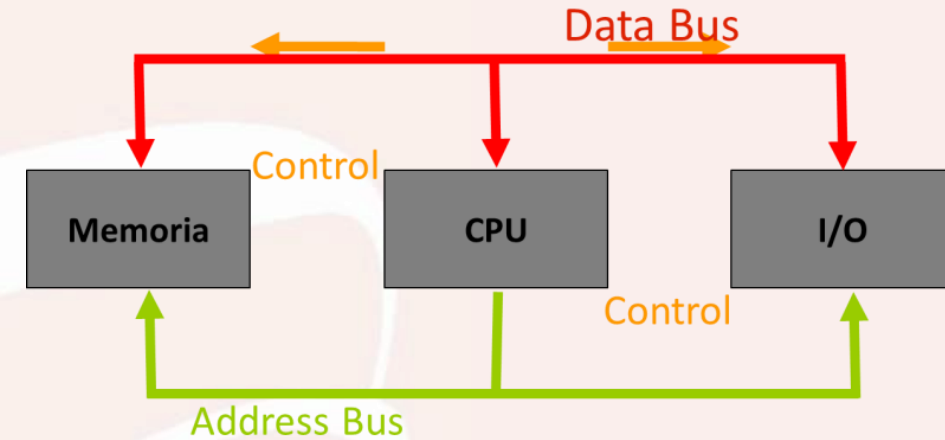
- operaciones sobre datos (algoritmos).

} CPU

- Entrada / Salida

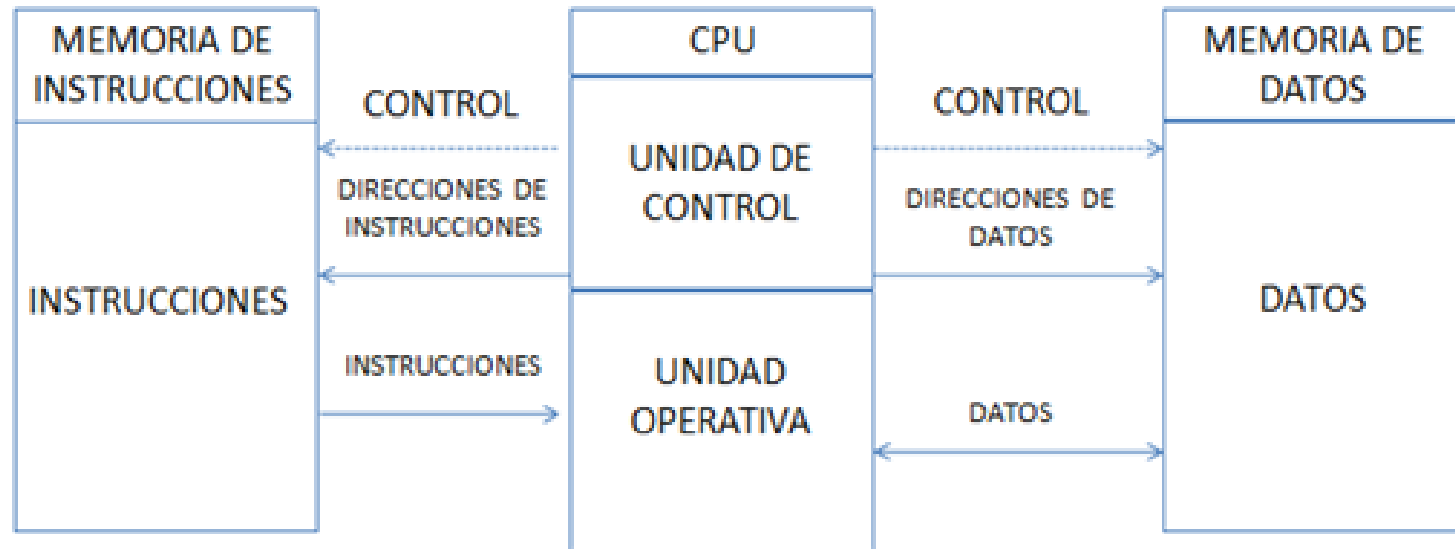
- Recibir nuevos datos.
 - Entregar resultados.

} Input/Output
(almacenamiento)

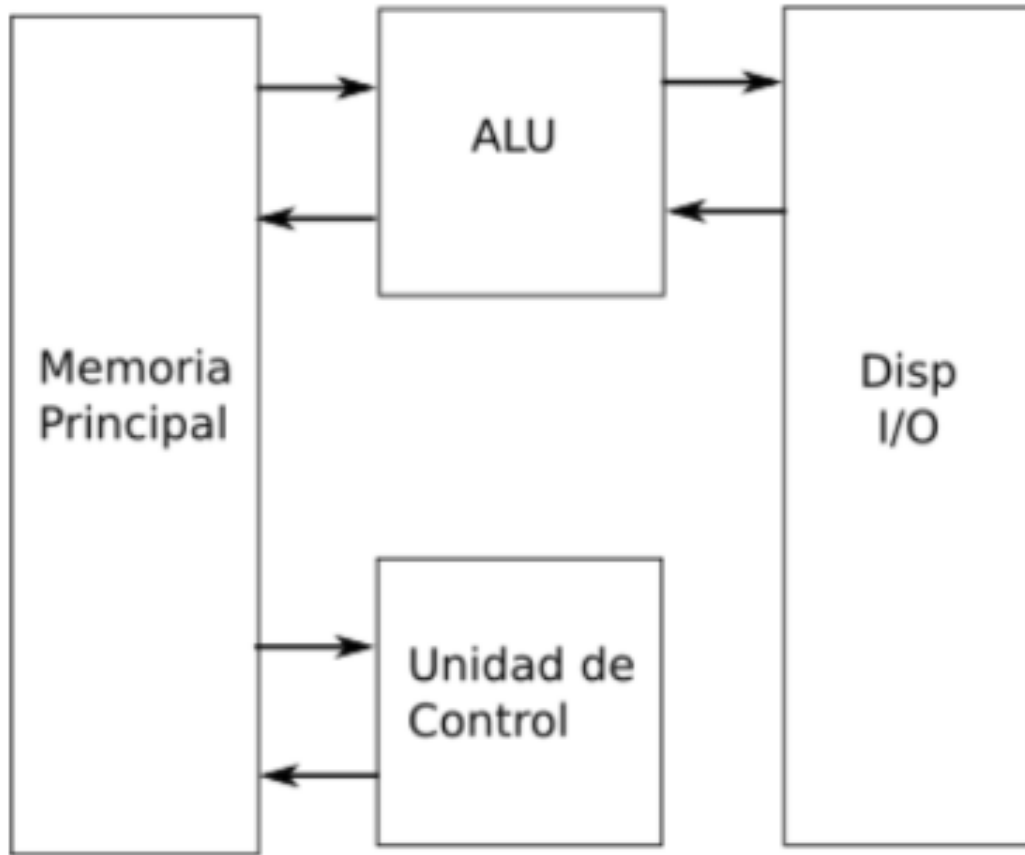


Arquitectura Harvard

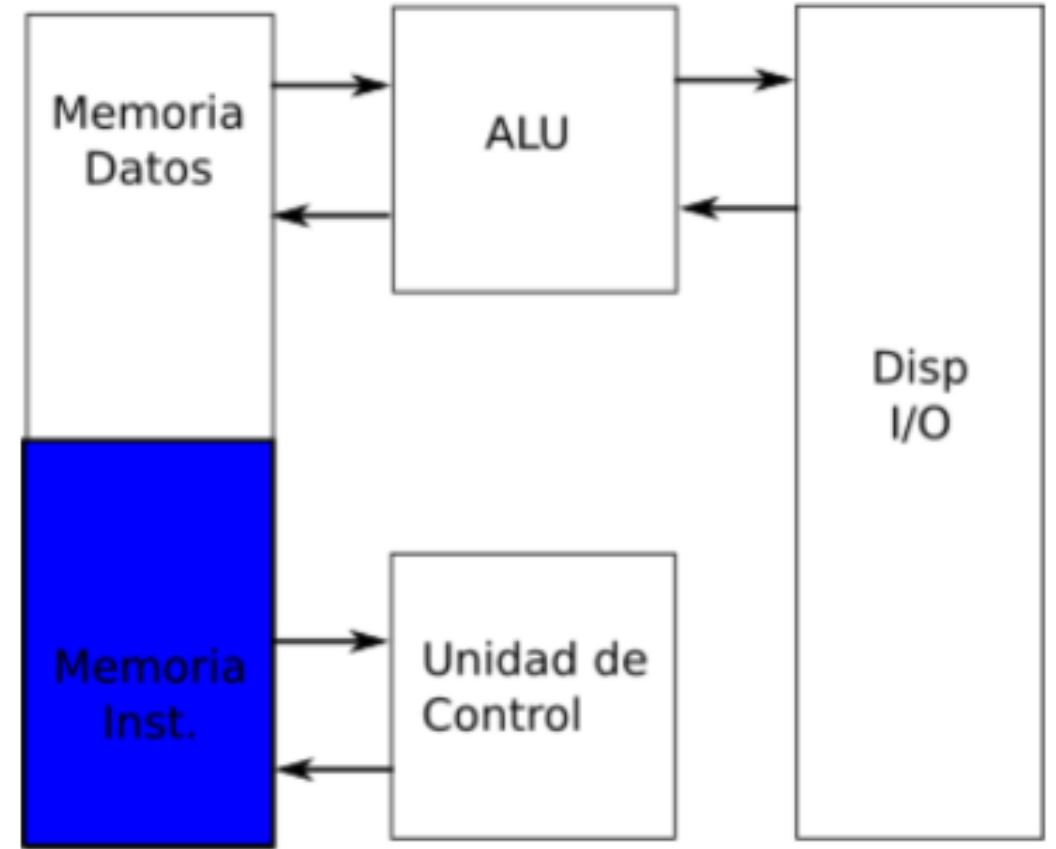
- Utiliza 2 memorias independientes
 - Datos
 - Instrucciones
- El término proviene de la computadora Harvard Mark I, que almacenaba las instrucciones en cintas perforadas y los datos en interruptores.



Organización de la memoria: arquitecturas de Von-Newman y Harvard



von-Neumann



Harvard

Arquitectura de von Neumann /Arquitectura Harvard

- Ventajas de la arquitectura de **von Neumann**

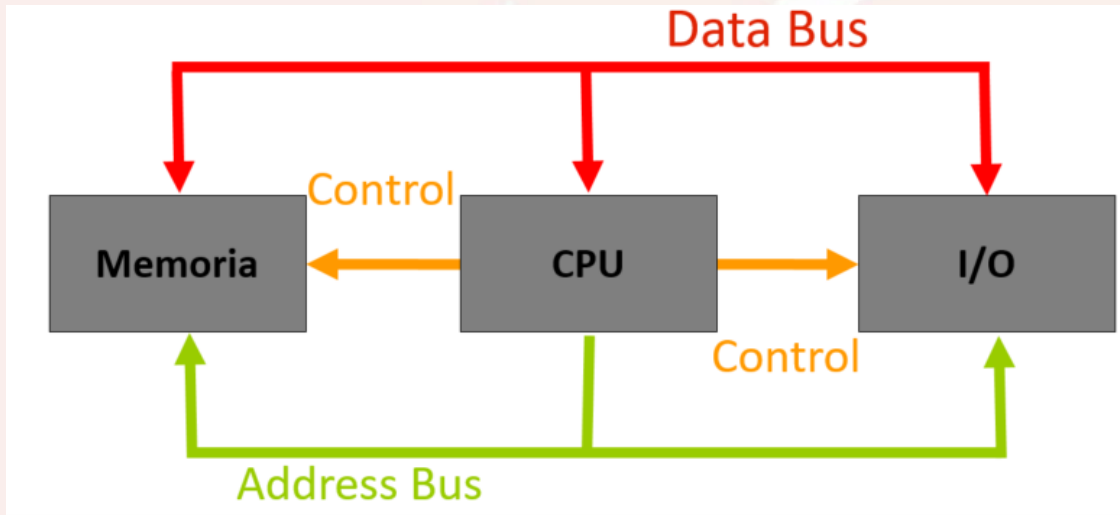
- Uso más eficiente de la memoria
 - Jerarquía de memoria no requiere estar dividida en 2
- Más simple arquitectónicamente
 - Un solo tipo de instrucciones para acceder a memoria
 - Un solo bus de datos y de direcciones
 - **Una misma forma de acceso a datos e instrucciones**
- Mayor flexibilidad
 - Útil para el sistema operativo
 - Paginación a disco
 - Código automodificante
 - **CPU con menos operaciones y mas flexible**

- Ventajas de la arquitectura **Harvard**

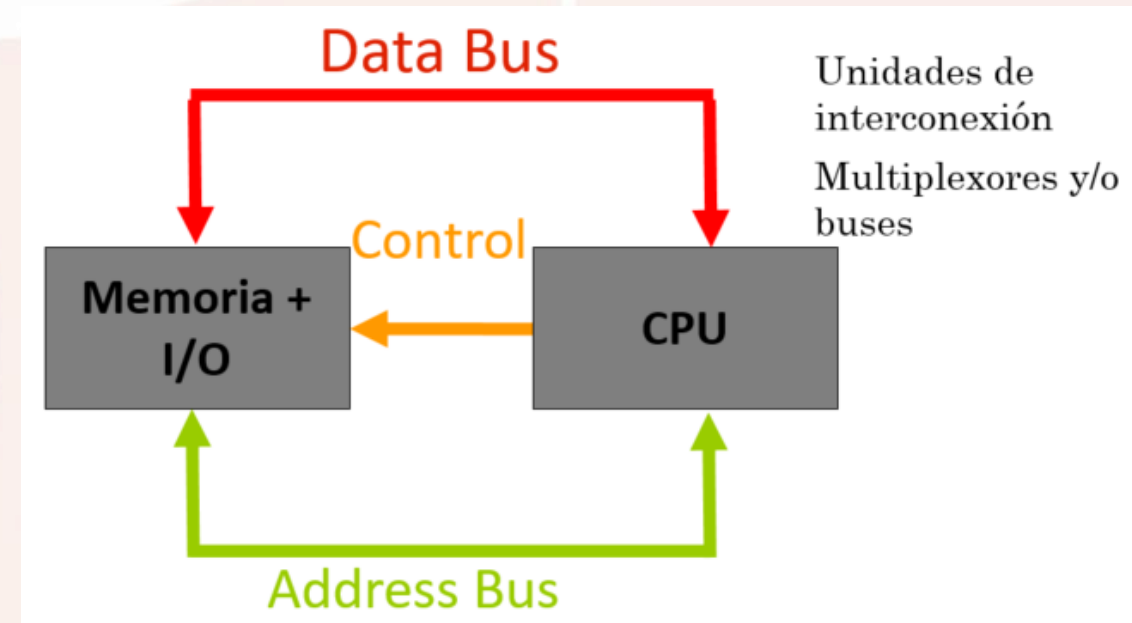
- Instrucciones y datos de distinto número de bits (ancho)
- Memorias de distinto tamaño
 - bits de direccionamiento distintos
- Memorias con distinta tecnología
- Disminuye el cuello de botella en el acceso a memoria
- El tiempo de acceso se mejora respecto a la arquitectura von Neumann donde programa y datos se traen a la CPU usando el mismo bus
- En los micro-controladores :
 - RAM para los datos
 - ROM para las instrucciones
- En las cache L1:
 - L1I
 - L1D

Estructura del computador: ¿Cómo comunicar CPU, Memoria e I/O?.

- E/S separada: buses separados



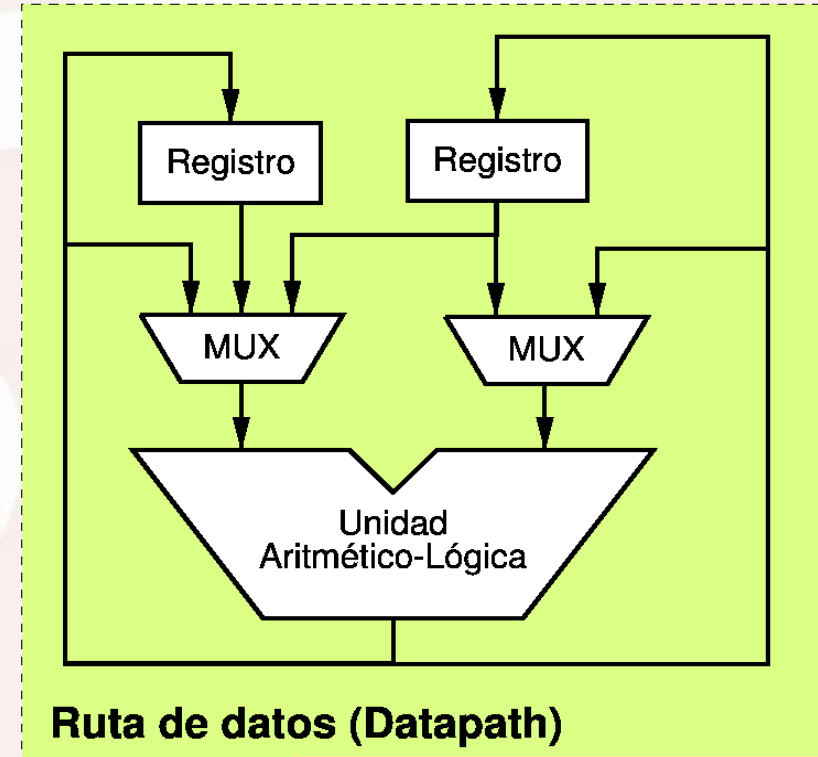
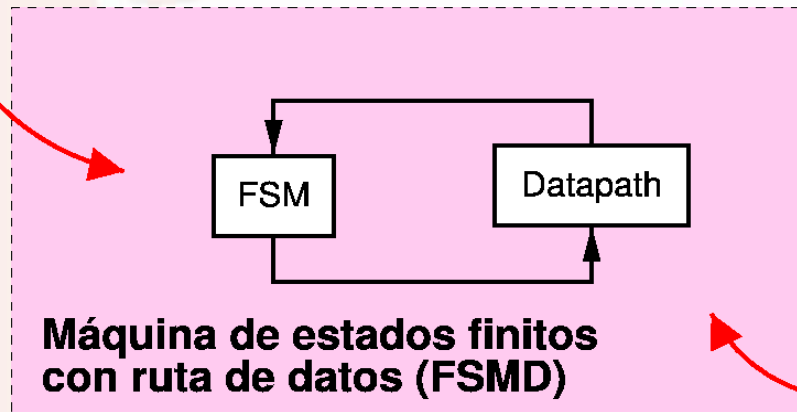
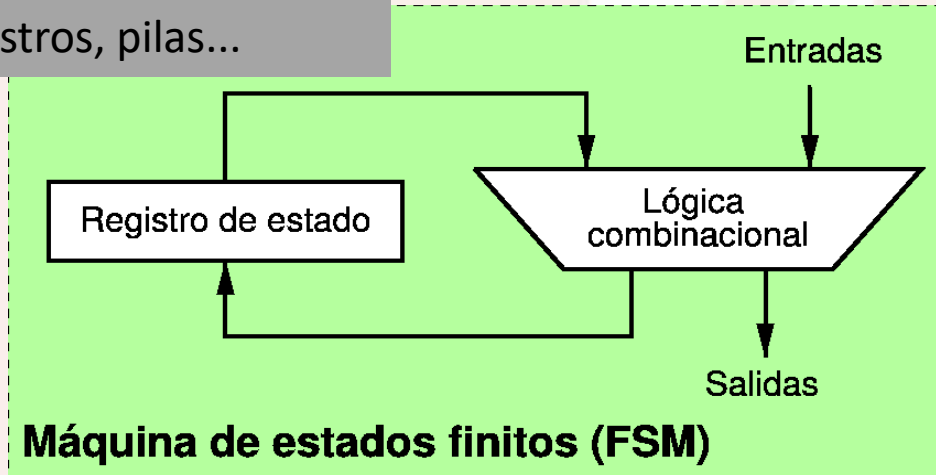
- E/S mapeada en memoria: Buses Compartidos



FSM + Datapath

Unidad de control

Responsable de ordenar cálculos
Secuenciador, registros, pilas...

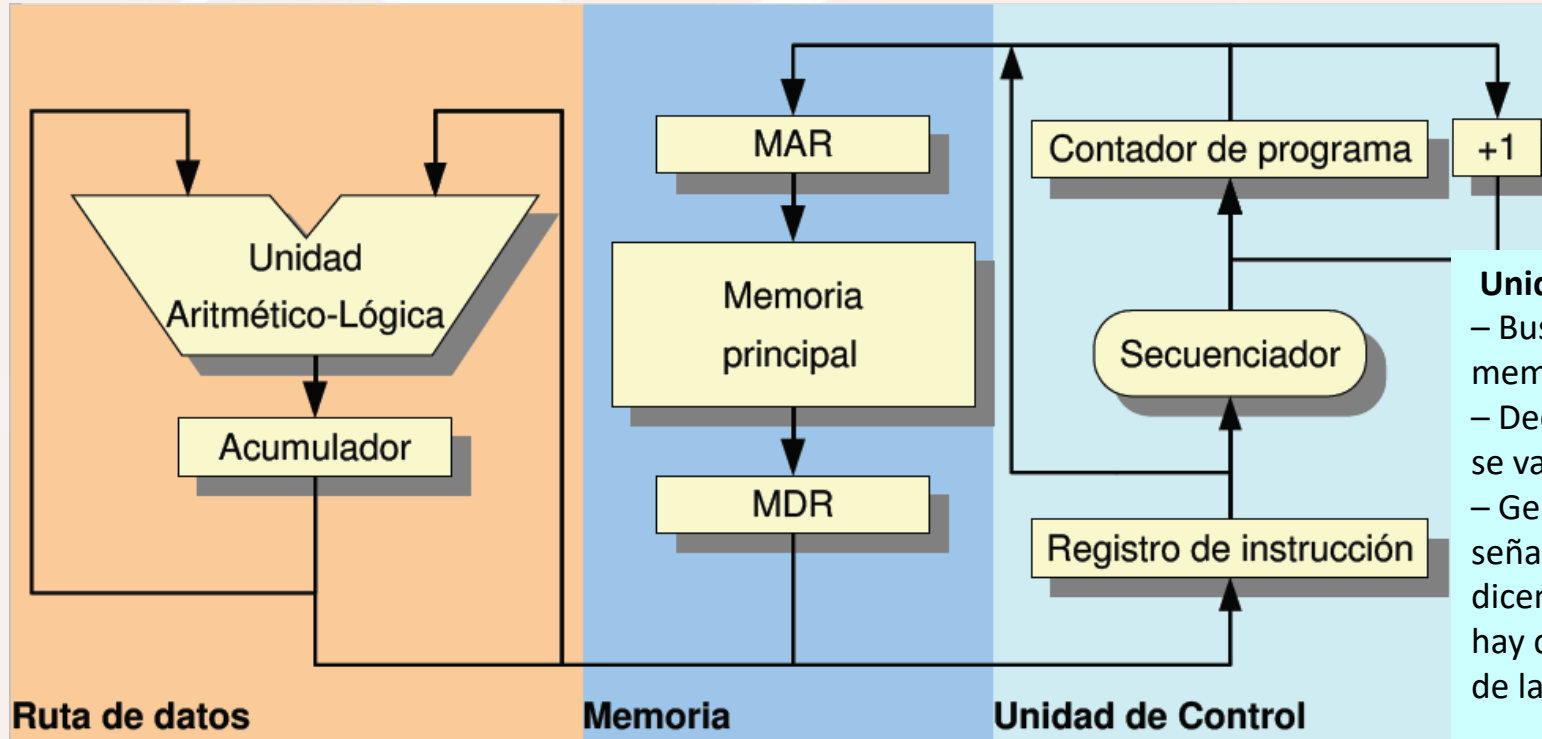


Ruta de datos

Responsable de realizar cálculos
Registros, unidades funcionales,
unidades de interconexión

Ejemplo de procesador

**Camino de datos
(Data path):**
unidades funcionales
que realizan las
operaciones de las
instrucciones.



Unidad de control:

- Busca las instrucciones de la memoria.
- Decodifica las instrucciones que se van a ejecutar.
- Genera los valores de las señales (señales de control) que dicen lo que hay que hacer para la ejecución de las instrucciones.

Ejecución de la instrucción

- Captura de operandos
- Operación
- Escritura de resultados

Búsqueda de la siguiente instrucción

Captación de la instrucción

- Leer dirección del Contador de Programa
- Incrementar el Contador de Programa

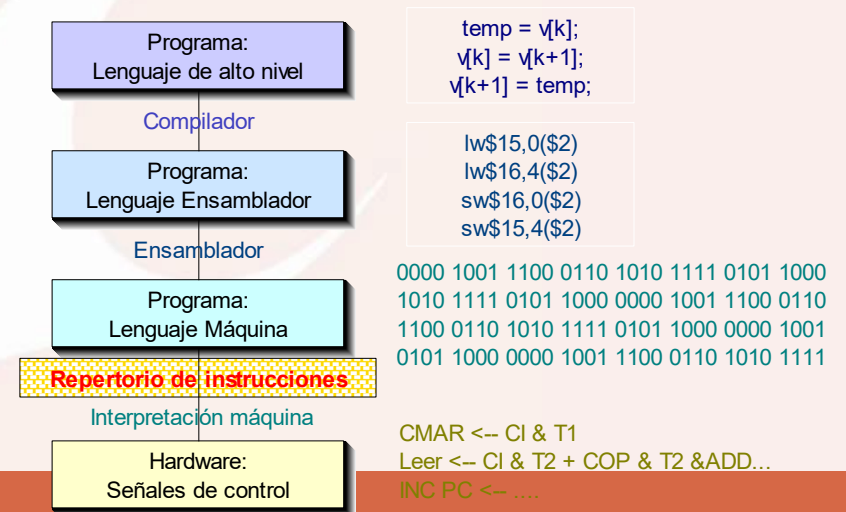
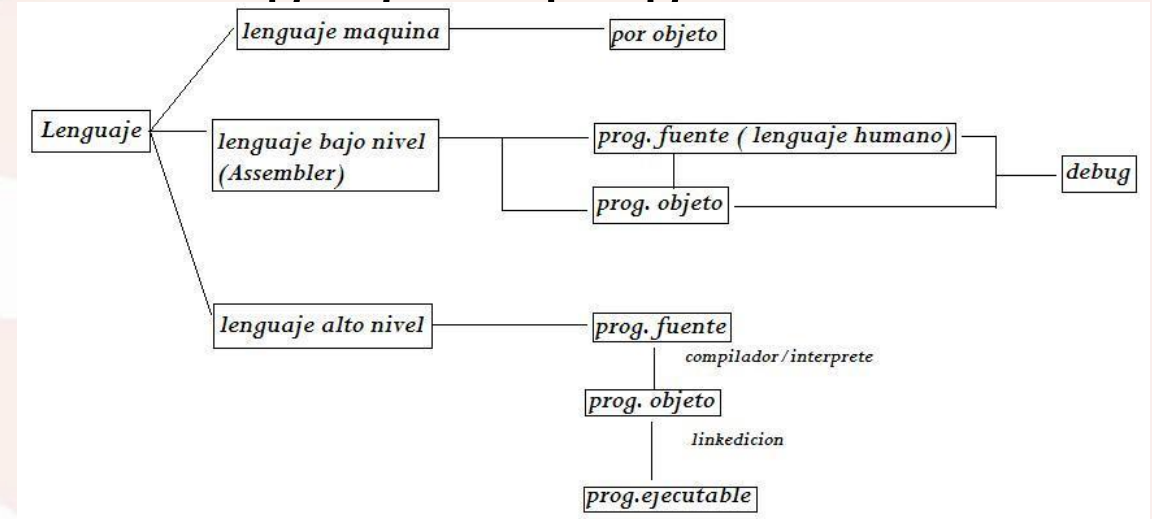
Decodificación de la instrucción

Software

- Programas e instrucciones

- **Lenguaje de programación**: símbolos y reglas para construir un programa
- Un **programa** es un conjunto ordenado de instrucciones que se dan al computador indicándoles las operaciones o tareas que se desea que realice
- **Repertorio de instrucciones**: conjunto de instrucciones distintas que puede ejecutar la CPU
- Una **instrucción** es un conjunto de símbolos que representa una orden de operaciones o tratamiento para el computador.
 - Las operaciones suelen realizarse con datos

- Lenguaje de programación



Esquema de funcionamiento de un computador: Hardware

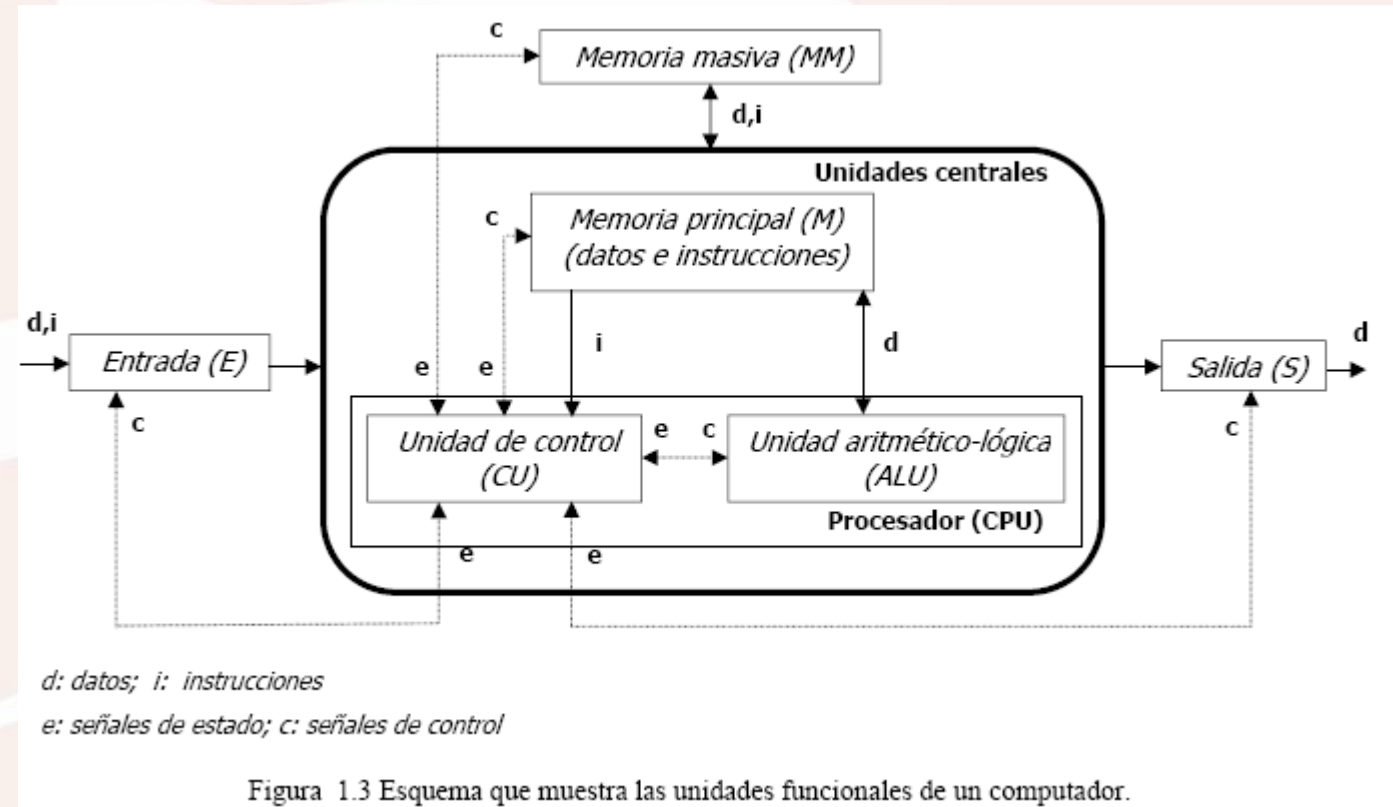
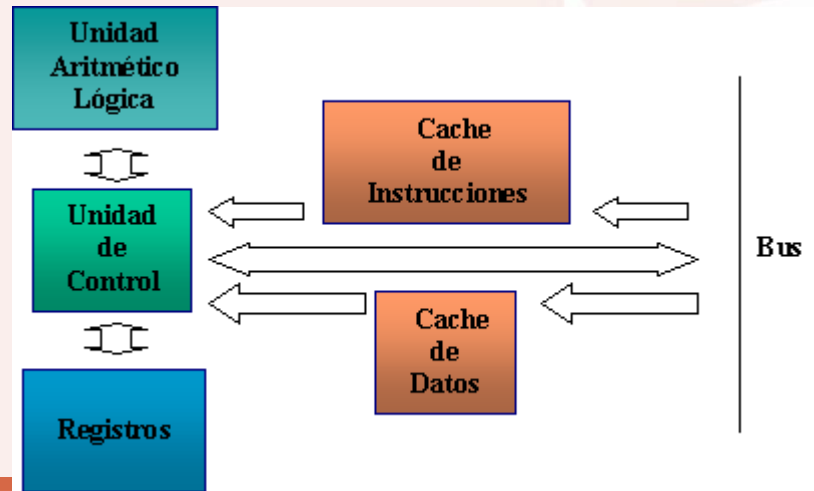
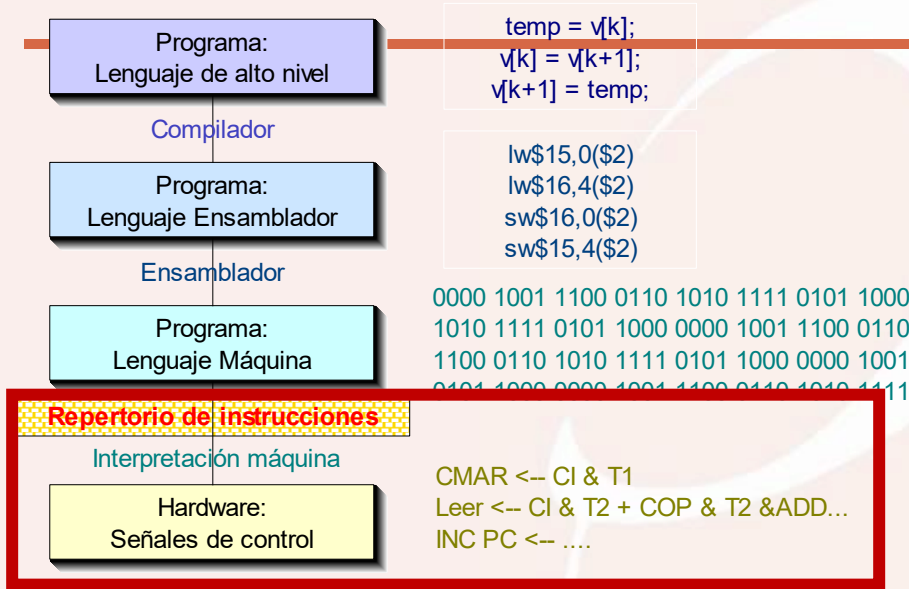
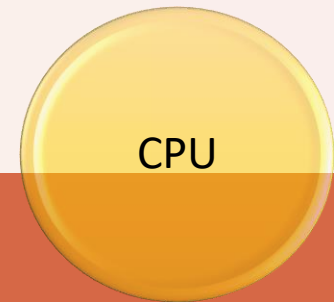


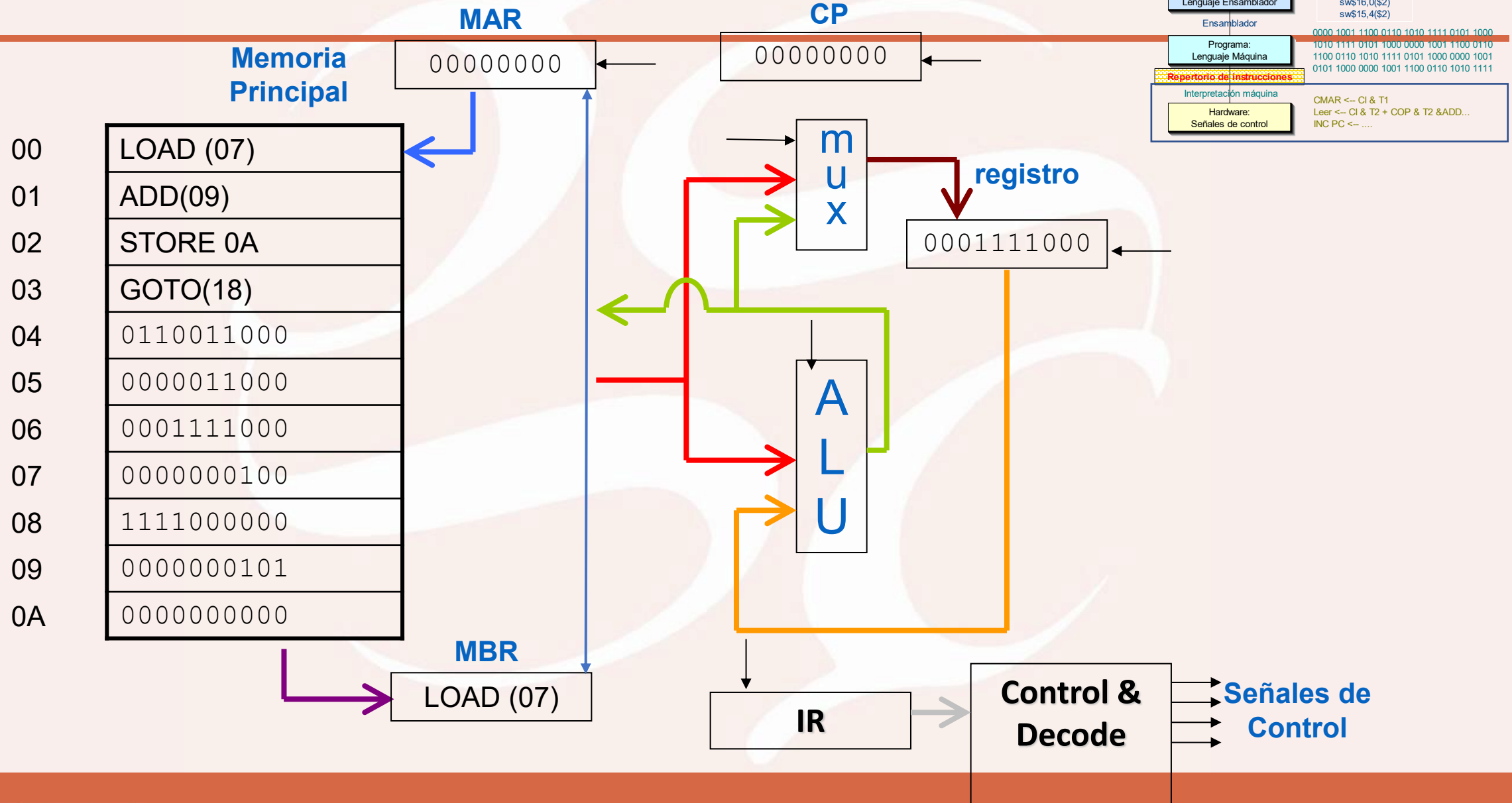
Figura 1.3 Esquema que muestra las unidades funcionales de un computador.

CPU

- La CPU del computador **busca, decodifica y ejecuta** las instrucciones de un programa
- Los dos componentes fundamentales de la CPU son la
 - **Unidad de Control:** Los componentes de la CPU ejecutan operaciones secuenciadas de acuerdo a las señales que proporciona su unidad de control
 - **El *datapath* o ruta de datos** consta
 - Unidad o unidades de procesamiento: UAL, FPU
 - Unidades de almacenamiento: registros
 - Interconectados por un bus de datos, que también está conectado a la memoria.
- Tareas:
 - Ejecutar Operaciones y Algoritmos
 - Controlar Flujo del Programa
 - Controlar Circuitos Internos



Ejemplo: implementación



Ejecutar operaciones

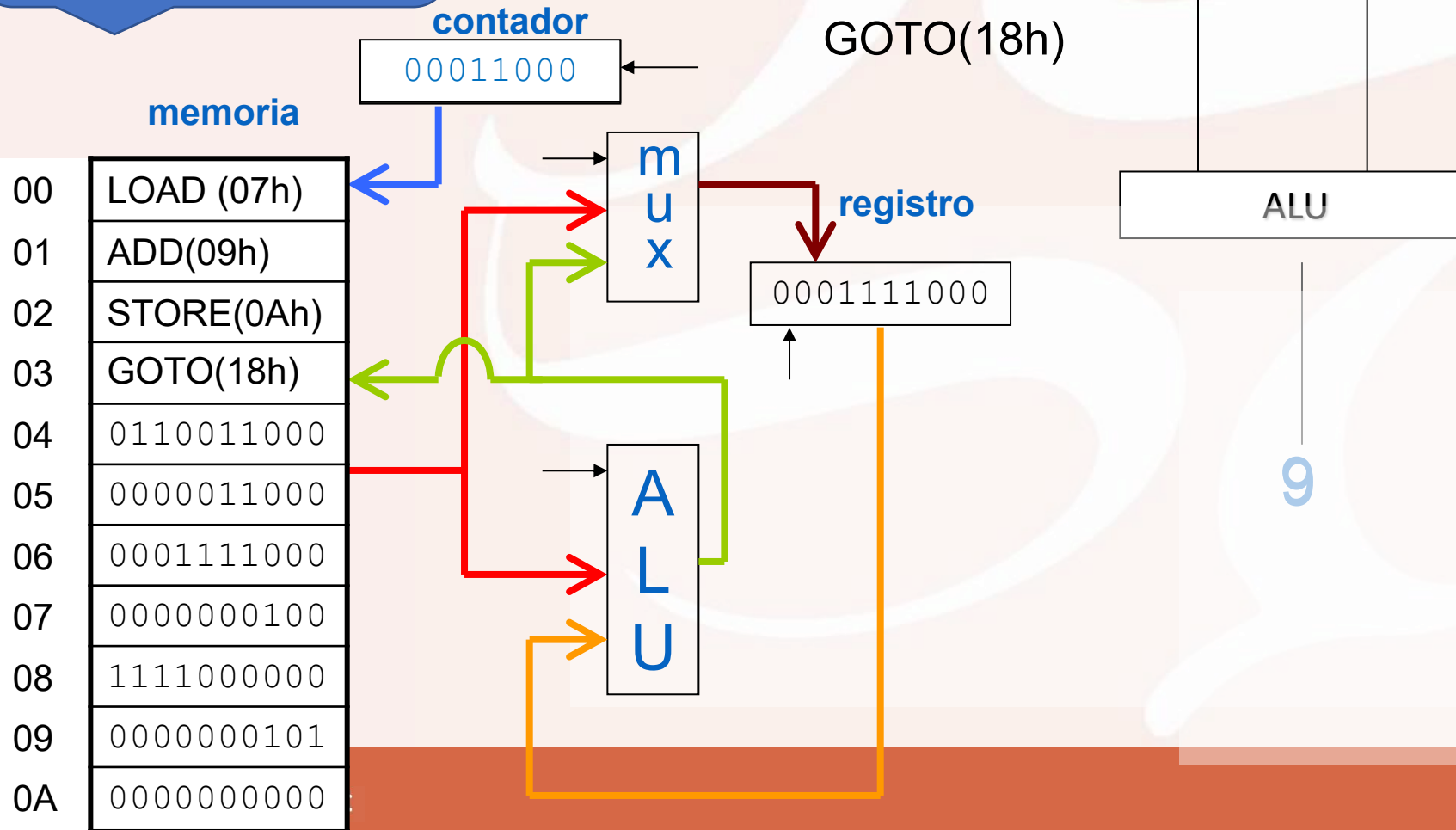
¿Qué posición de memoria queda alterada?
¿Con qué valor?

LOAD (07h)
ADD(09h)
STORE(0Ah)
GOTO(18h)

Memoria

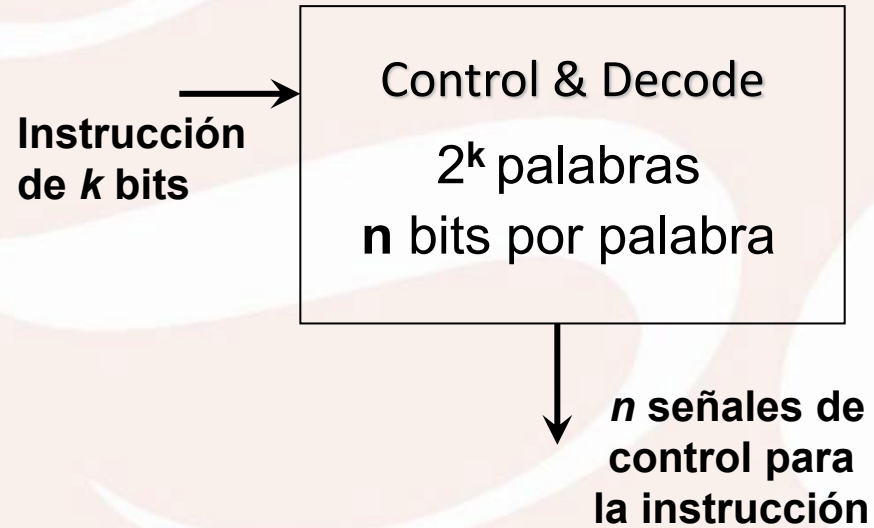
00	1101110111	← LOAD (07h)
01	1100011001	ADD (09h)
02	1110001010	STORE(0Ah)
03	1110000000	GOTO(18h)
04	0110011000	
05	0000011000	
06	0001111000	
07	0000000100	4
08	1111000000	← 5
09	0000000101	
0A	0000001001	

R.Mitnik



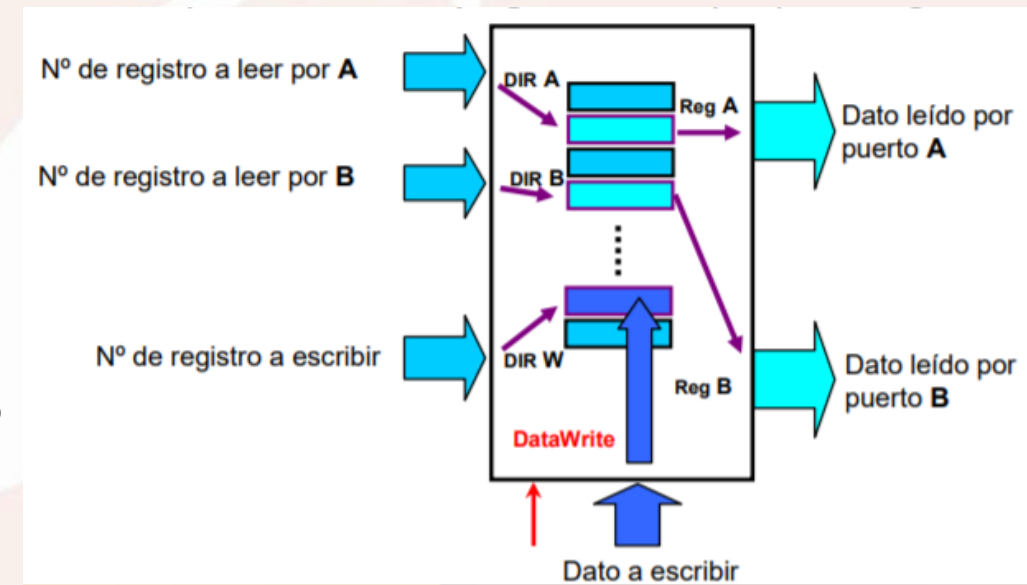
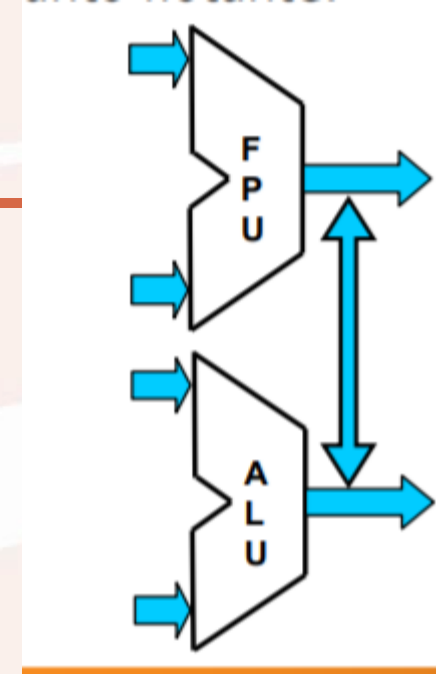
Control y decodificación

- Control & Decodificación procesa las instrucciones.
 - Unidad de control cableada / Unidad de control microprogramada (ROM)
 - Espacio de Direccionamiento determina cantidad de instrucciones
 - Las salidas son señales de control para ejecutar una instrucción.



Camino de datos

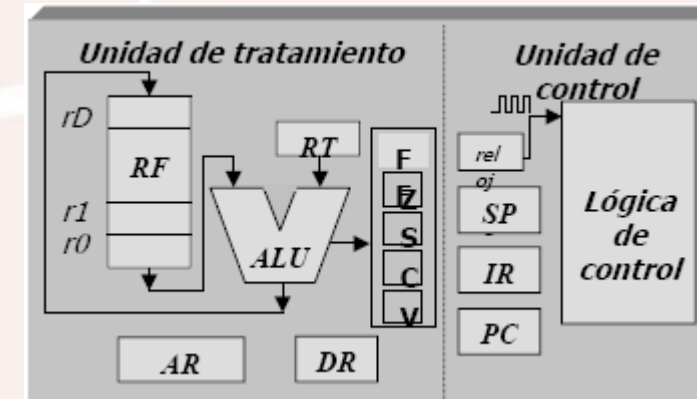
- **Unidad aritmético lógica (UAL)** Realiza las operaciones aritméticas y lógicas requeridas por el programa en ejecución.
 - La operación a efectuar se establece con las señales de control.
 - Operación:
 - $BS \leftarrow BA + BS$
 - suma
 - Señales de control: 001
- **Unidad de coma flotante (FPU) o “Floating-Point Unit”**: realiza las operaciones aritméticas con operandos de punto flotante.
- **Banco de registros**: Estructura que aglutina un número pequeño de registros .
 - Cada registro contiene un dato que puede ser operado por la ALU o FPU



Registros del procesador

- Un procesador incluye:
 - **Registros visibles por el Usuario:**
 - Pueden referenciarse en las instrucciones.
 - Permiten al programador minimizar las referencias a la memoria principal mediante la optimización del uso de registros
 - Se clasifican en:
 - Uso general.
 - Datos.
 - Direcciones.
 - Códigos de Condición
 - **Registros de Control y Estado**
 - Usados por el procesador para control de la operatividad del propio procesador
 - Usados por las rutinas del sistema operativo para controlar la ejecución de los programas
 - **Registros no visibles**
 - Registros temporales

Registros de la CPU: ¿Cuántos registros de propósito general?



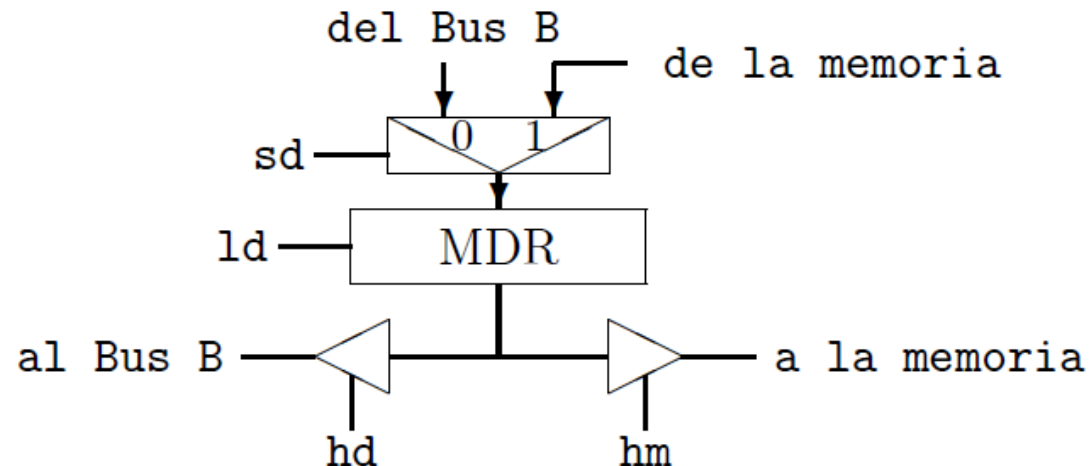
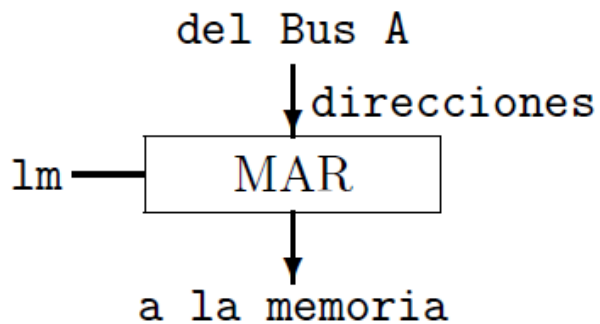
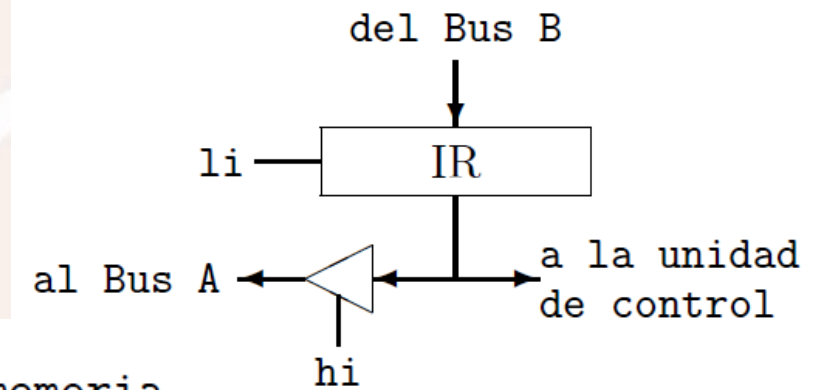
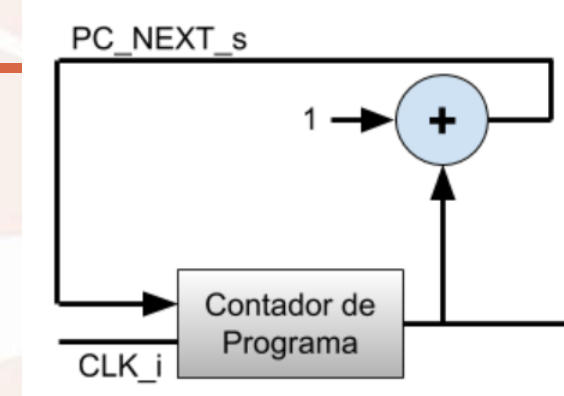
¿Ejemplos?

Registros de la CPU: ¿Cuántos registros de propósito general?

- Generalmente entre 8 y 32
 - Pocos registros
 - Requiere demasiados accesos a memoria.
 - Muchos registros
 - Reducen las referencias a memoria
 - CPU más compleja.

Registros de la CPU.

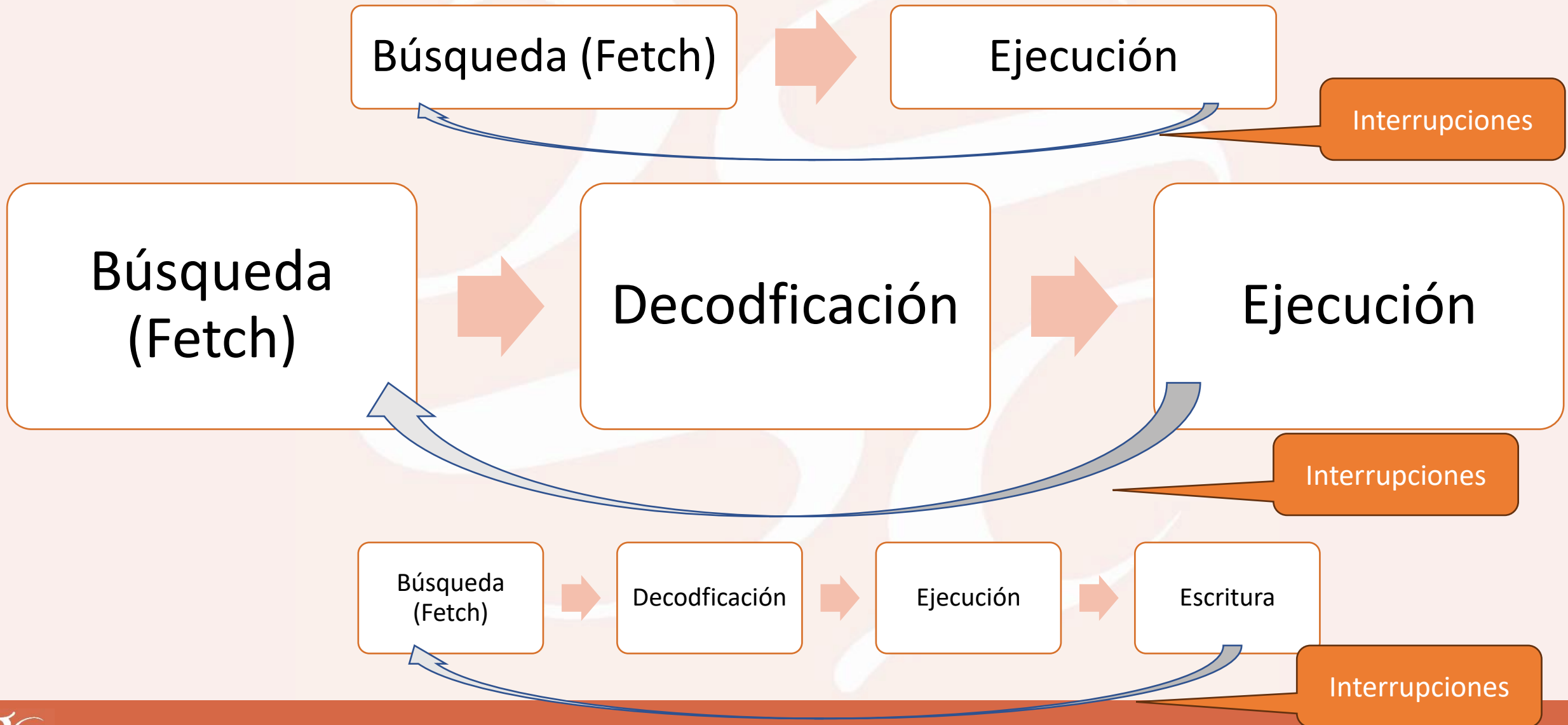
- Registros
 - Controlar el funcionamiento de la CPU
 - PC = *Program Counter*: contiene la dirección de la siguiente instrucción a ejecutar
 - IR = *Instruction Register*: contiene la última instrucción captada
 - MAR = *Memory Address Register*: contiene la dirección de una posición de memoria.
 - MBR = *Memory Buffer Register*: contiene la palabra de datos a escribir en memoria, o la palabra leída más recientemente
 - Registro de control / estado



Registros de la CPU: Registros Control/Estado.

- PSW = *Program Status Word*:
 - **Signo**: contiene el bit de signo del resultado de última operación
 - **Cero**: puesto a uno cuando el resultado es 0
 - **Acarreo**: se activa si en la suma hay acarreo o en la resta hay un adeudo del bit más significativo
 - **Igual**: se activa si el resultado de una comparación lógica es la igualdad
 - **Desbordamiento**: Usado para indicar desbordamiento aritmético
- Control
 - **Interrupciones**: usado para permitir o inhabilitar interrupciones
 - **Supervisor**: indica si la CPU funciona en modo supervisor o usuario. Únicamente en modo supervisor se pueden ejecutar ciertas instrucciones privilegiadas y se puede acceder a ciertas áreas de memoria

Secuencia básica de ejecución de instrucciones



Fases en la ejecución de una instrucción

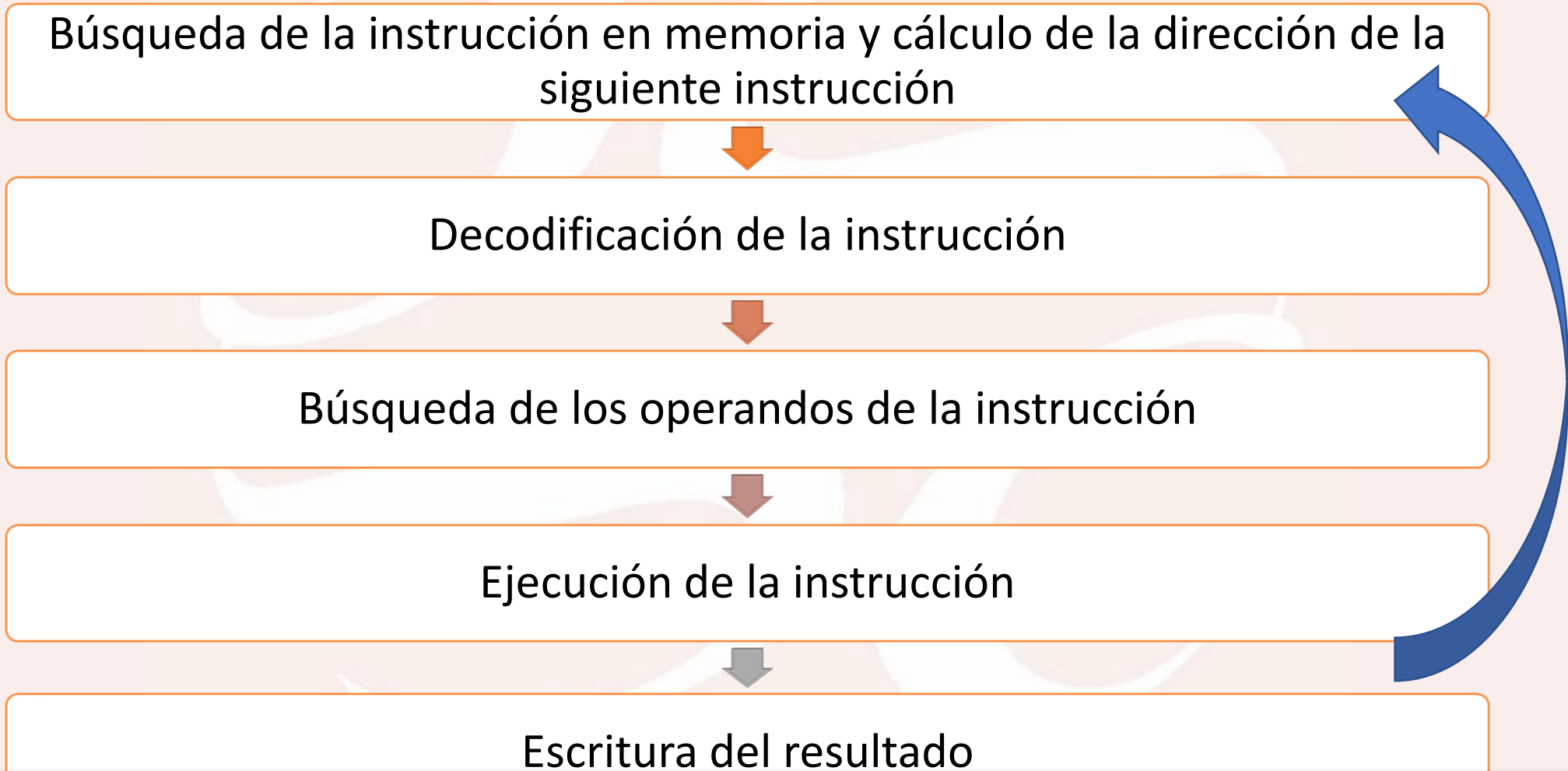
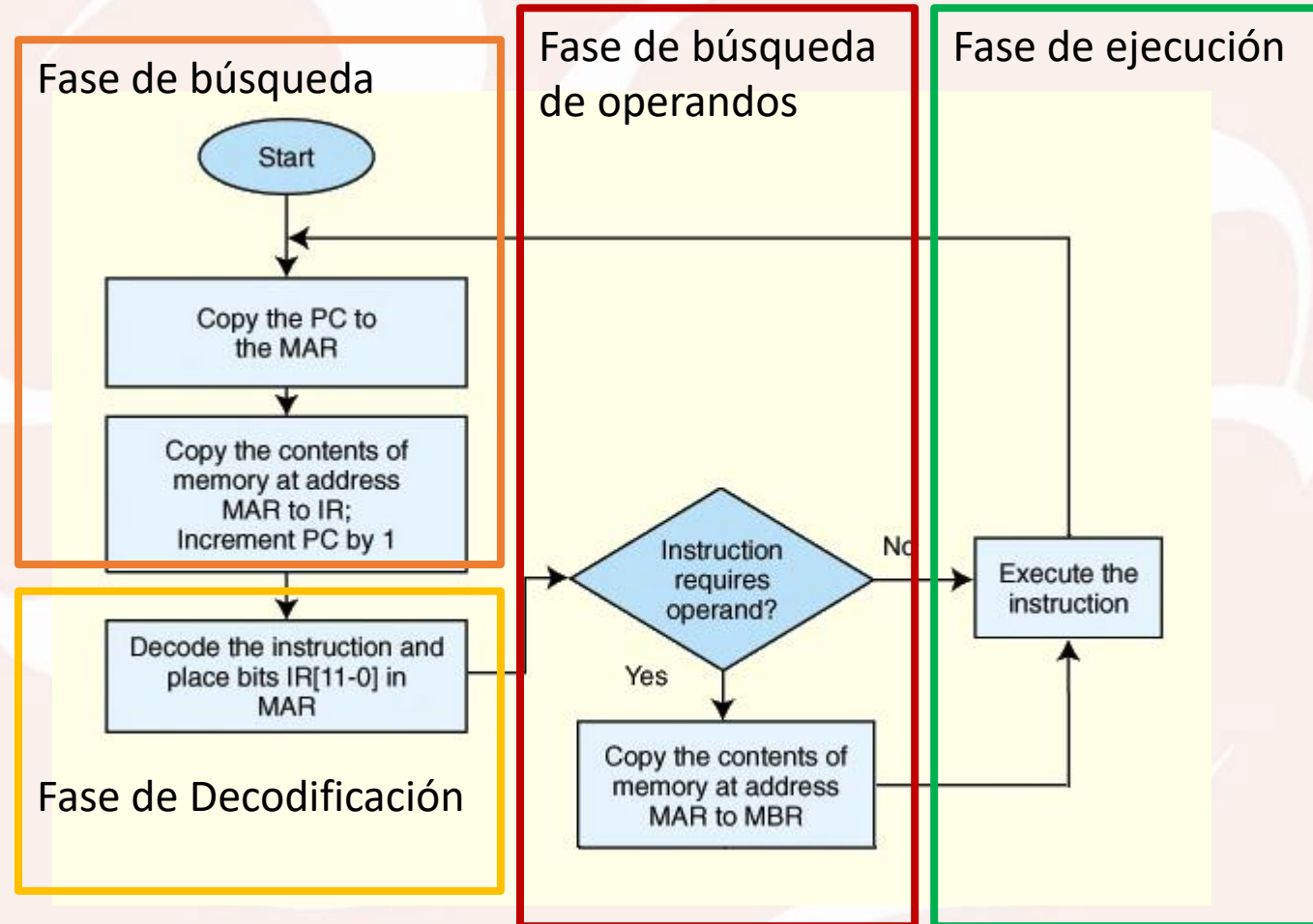


Diagrama de Estados: representa una máquina de estados finita (**FSM**).

Este ejemplo de máquina interpreta instrucciones máquina y controla el uso de los elementos del procesador (registros, memoria, ALU, ...)



Procesamiento de las instrucciones



El Procesador

Fases de ejecución de instrucciones

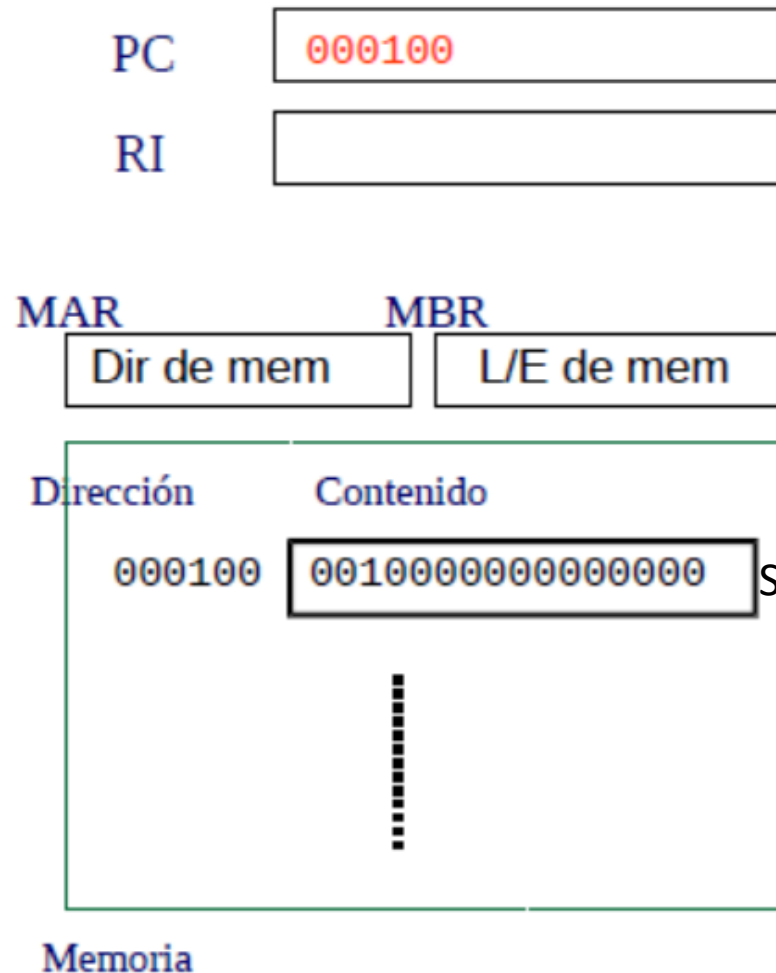
■ Lectura de la instrucción (ciclo de *fetch*)

- ❑ $MAR \leftarrow PC$
- ❑ Lectura
- ❑ $MBR \leftarrow \text{Memoria}$
- ❑ $PC \leftarrow PC + 1$
- ❑ $RI \leftarrow MBR$

■ Decodificación de la instrucción

■ Ejecución de la instrucción

■ Volver a *fetch*



Se carga en R0 el valor 0

El Procesador

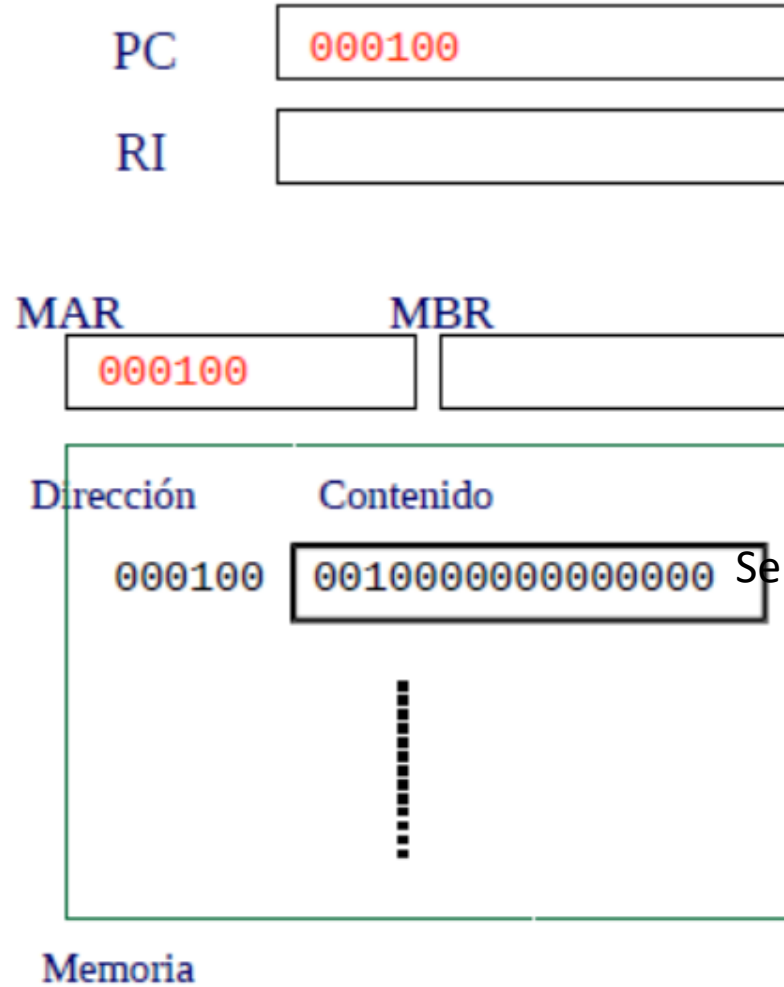
Fases de ejecución de instrucciones

■ Lectura de la instrucción (ciclo de *fetch*)

- ☐ $\text{MAR} \leftarrow \text{PC}$
- ☐ Lectura
- ☐ $\text{MBR} \leftarrow \text{Memoria}$
- ☐ $\text{PC} \leftarrow \text{PC} + 1$
- ☐ $\text{RI} \leftarrow \text{MBR}$

■ Decodificación de la instrucción

- Ejecución de la instrucción
- Volver a *fetch*



Se carga en R0 el valor 0

El Procesador

Fases de ejecución de instrucciones

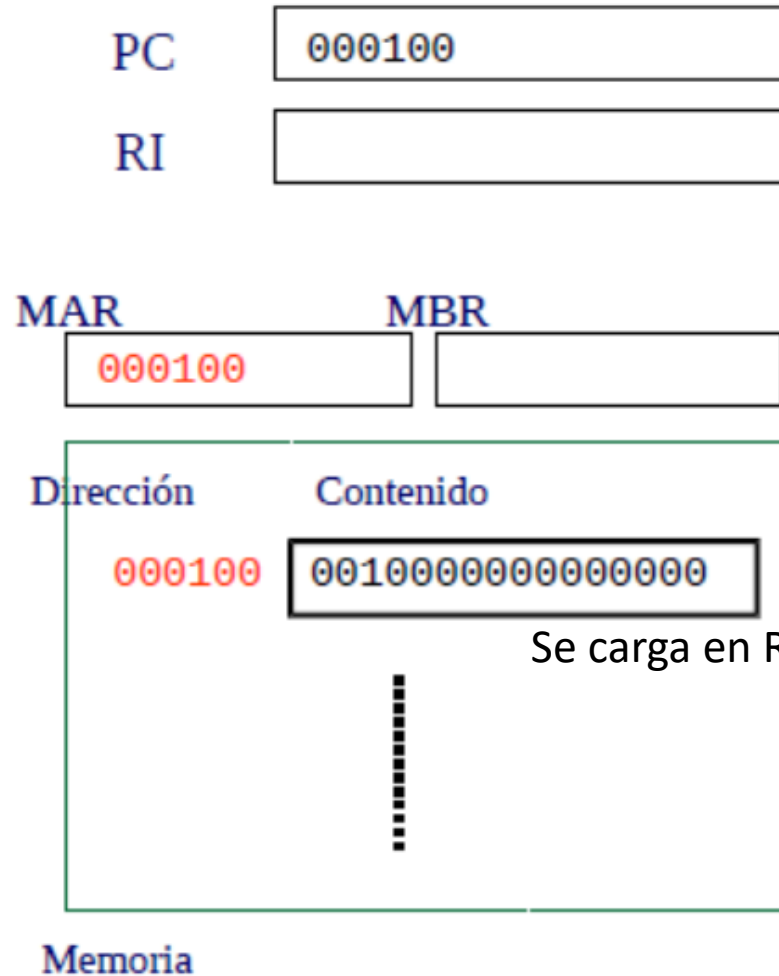
■ Lectura de la instrucción (ciclo de *fetch*)

- ☐ $MAR \leftarrow PC$
- ☐ **Lectura**
- ☐ $MBR \leftarrow \text{Memoria}$
- ☐ $PC \leftarrow PC + 1$
- ☐ $RI \leftarrow MBR$

■ Decodificación de la instrucción

■ Ejecución de la instrucción

■ Volver a *fetch*



Se carga en R0 el valor 0

El Procesador

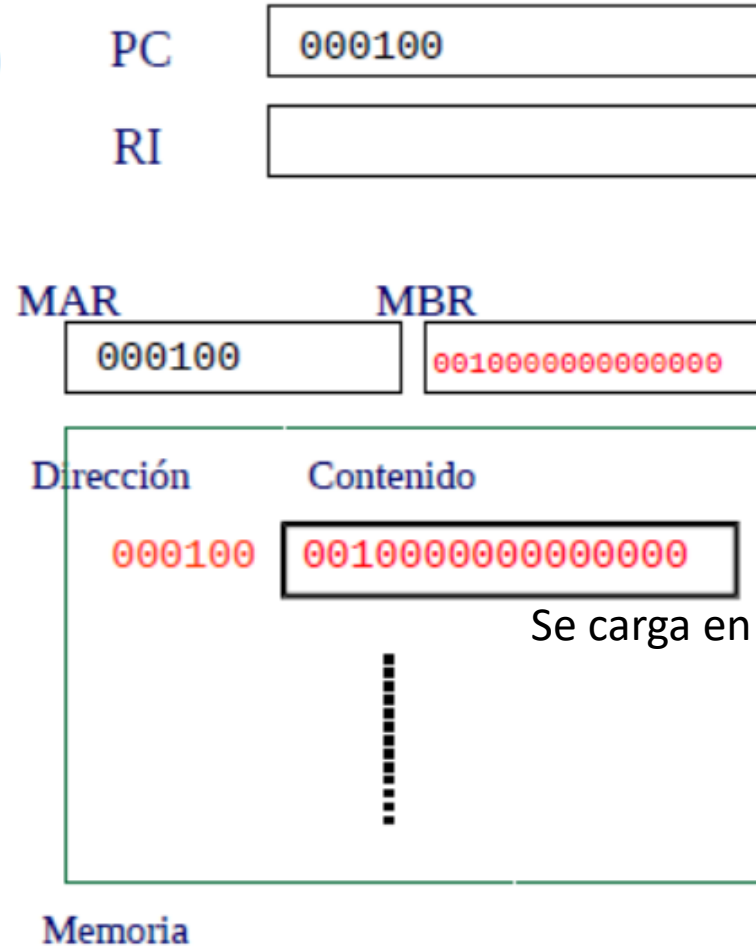
Fases de ejecución de instrucciones

■ Lectura de la instrucción (ciclo de *fetch*)

- ❑ $MAR \leftarrow PC$
- ❑ Lectura
- ❑ $MBR \leftarrow \text{Memoria}$
- ❑ $PC \leftarrow PC + 1$
- ❑ $RI \leftarrow MBR$

■ Decodificación de la instrucción

- Ejecución de la instrucción
- Volver a *fetch*



El Procesador

Fases de ejecución de instrucciones

■ Lectura de la instrucción (ciclo de *fetch*)

□ $MAR \leftarrow PC$

□ Lectura

□ $MBR \leftarrow \text{Memoria}$

□ $PC \leftarrow PC + 1$

□ $RI \leftarrow MBR$

■ Decodificación de la instrucción

■ Ejecución de la instrucción

■ Volver a *fetch*

PC

000101

RI

MAR

000100

MBR

0010000000000000

Dirección

000100

Contenido

0010000000000000

Se carga en R0 el valor 0

Memoria

El Procesador

Fases de ejecución de instrucciones

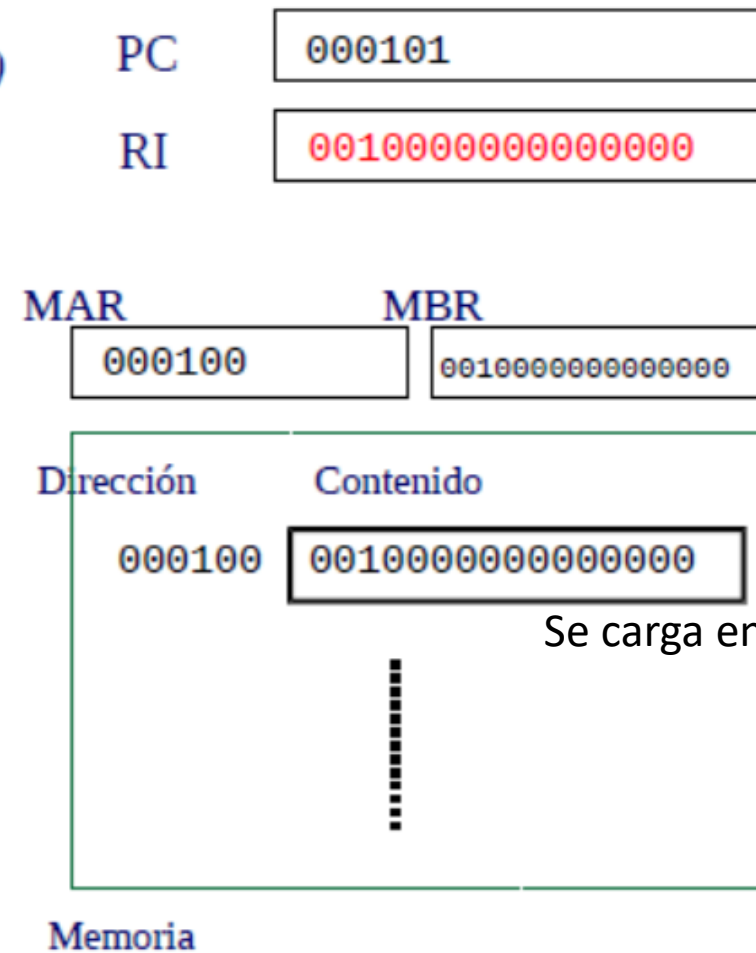
■ Lectura de la instrucción (ciclo de *fetch*)

- $MAR \leftarrow PC$
- Lectura
- $MBR \leftarrow \text{Memoria}$
- $PC \leftarrow PC + 1$
- **$RI \leftarrow MBR$**

■ Decodificación de la instrucción

■ Ejecución de la instrucción

■ Volver a *fetch*



El Procesador

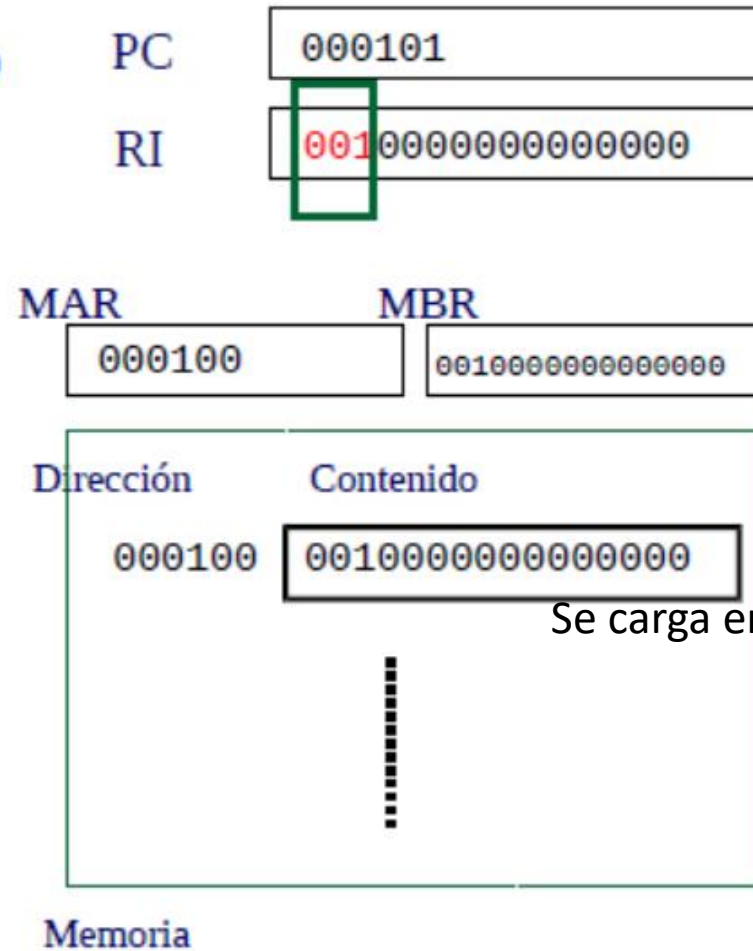
Fases de ejecución de instrucciones

■ Lectura de la instrucción (ciclo de *fetch*)

- $MAR \leftarrow PC$
- Lectura
- $MBR \leftarrow \text{Memoria}$
- $PC \leftarrow PC + 1$
- $RI \leftarrow MBR$

■ Decodificación de la instrucción

- Ejecución de la instrucción
- Volver a *fetch*



El Procesador

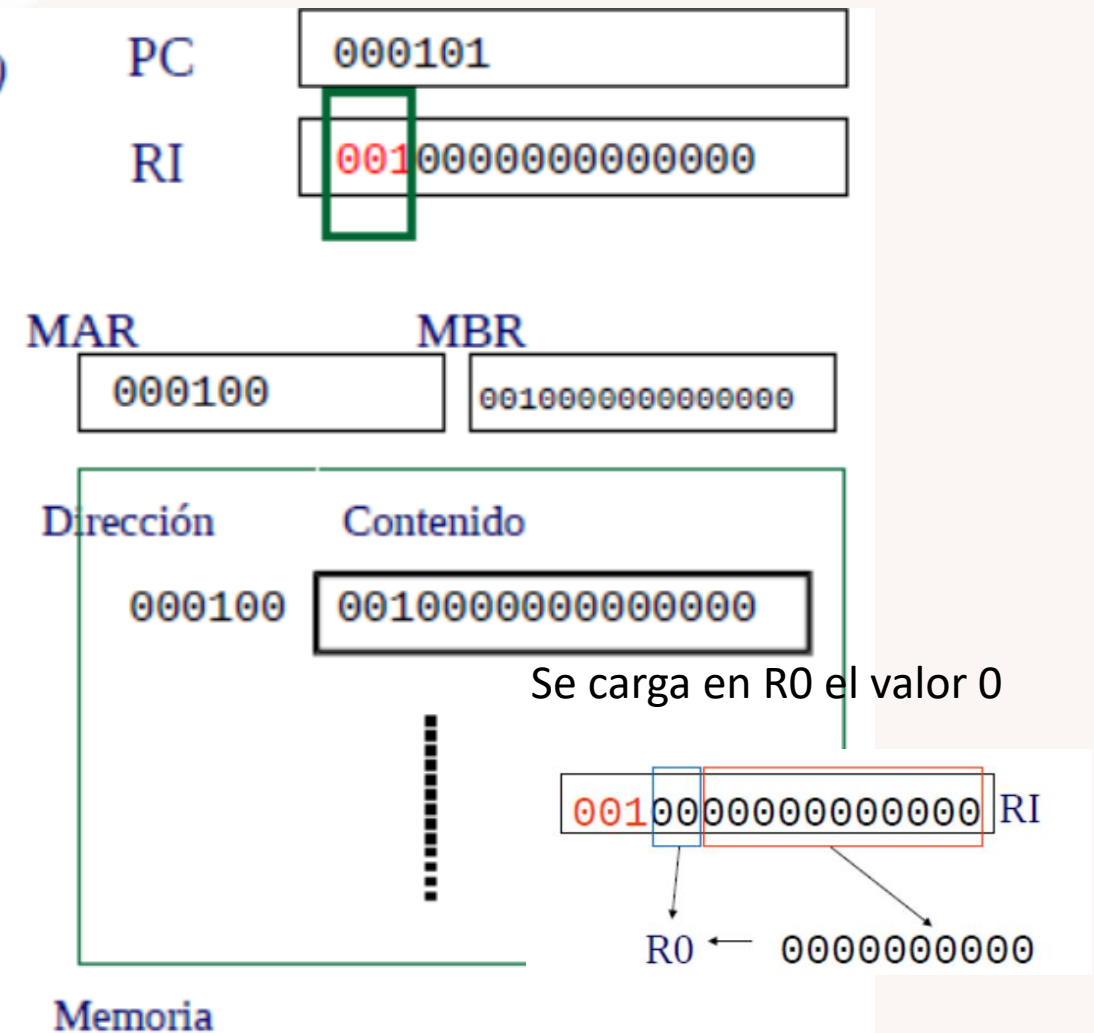
Fases de ejecución de instrucciones

■ Lectura de la instrucción (ciclo de *fetch*)

- $MAR \leftarrow PC$
- Lectura
- $MBR \leftarrow \text{Memoria}$
- $PC \leftarrow PC + 1$
- $RI \leftarrow MBR$

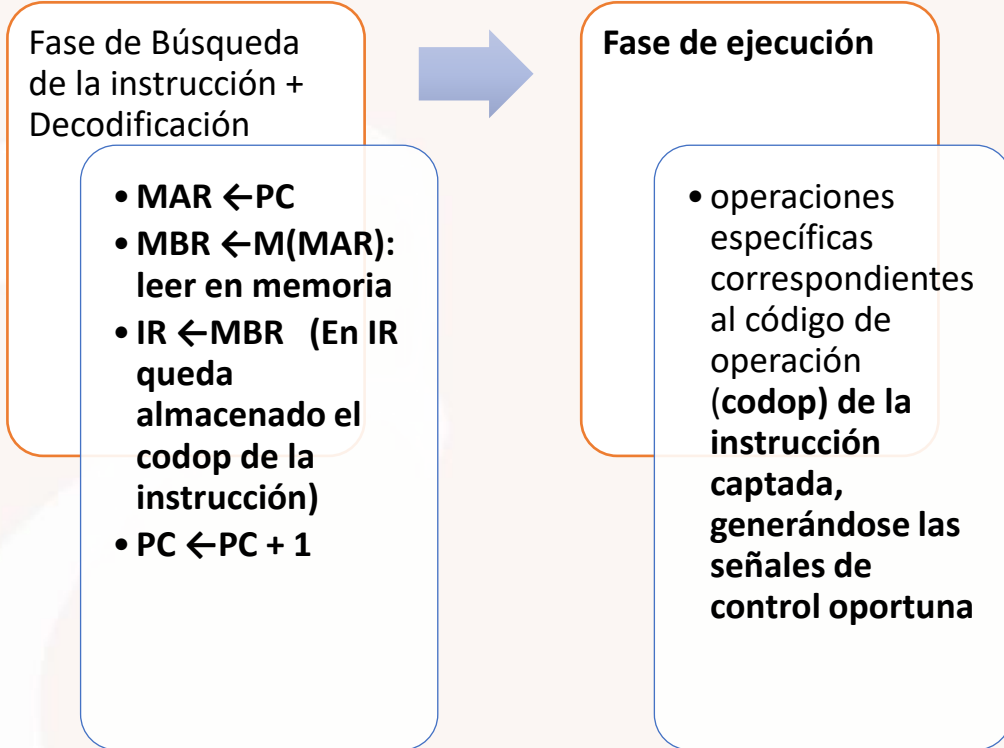
■ Decodificación de la instrucción

- Ejecución de la instrucción
- Volver a *fetch*



Funcionamiento interno de un computador

- La ejecución de cualquier instrucción se realiza en dos fases:
 - Fase de captación y decodificación de la instrucción.
 - Fase de ejecución



La ejecución de una instrucción lleva consigo por lo menos un acceso a memoria (para captar la instrucción), pudiendo efectuar accesos adicionales en la fase de ejecución (captación de operandos, memorización de resultados, etc.)

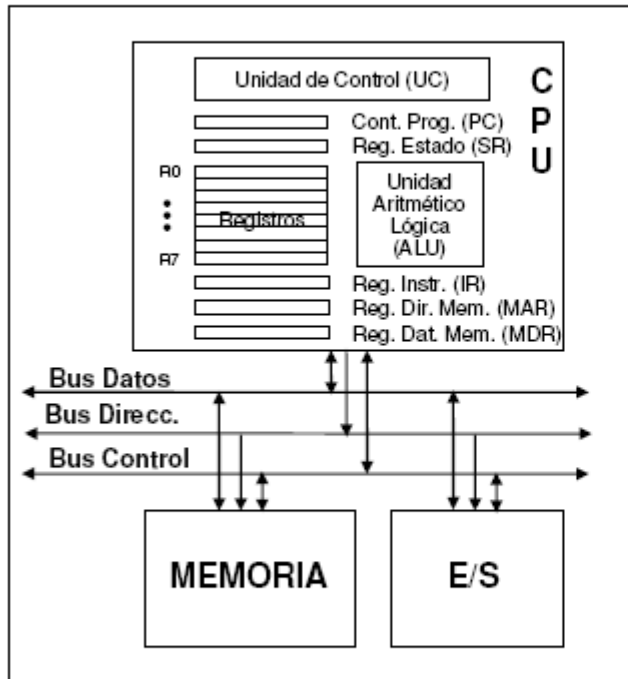
Fase de Búsqueda de la instrucción

-
- El diagrama ilustra la arquitectura de un sistema de procesamiento de datos, dividido en tres secciones principales: Unidad de tratamiento, Unidad de control y Memoria.
- Unidad de tratamiento:** Contiene un Registro de Datos (RD) con direcciones rD, r1 y r0. Los datos de r1 y r0 se envían a la ALU (Arithmetic Logic Unit). La ALU también recibe un registro de Constantes (RT) y envía su resultado a un Registro de Resultados (RR). El RR está conectado a un Registro de Estado (F) que incluye bits de bandera: E, Z, S, C y V. El resultado de la ALU también se envía a un Registro de Direcciones (DR).
- Unidad de control:** Incluye un reloj (relaj), un Registro de Pila (SP), un Registro de Instrucción (IR) que contiene el valor BC73, y el Program Counter (PC) que contiene el valor 100E. La Lógica de control recibe el reloj y envía señales a la ALU y al Registro de Instrucción.
- Memoria:** Muestra una tabla de direcciones y sus valores hexadecimales:
- | Dirección | Valor Hexadecimal |
|-----------|-------------------|
| 100A | A73C |
| 100B | 7C4D |
| 100C | 65AD |
| 100D | BC73 |
| 100E | 1000 |
| 100F | 3A02 |
| 1010 | 7BC0 |
- Los tres buses de comunicación (Bus de direcciones, Bus de datos y Bus de control) conectan las tres secciones del sistema.

Resumen:

Estructura básica de un computador

Esquema



Módulos básicos

CPU (Unidad Central de Proceso)

- Realiza la ejecución de las instrucciones
- Bloques:
 - ⇒ Unidad de proceso (ruta de datos): ALU, banco de registros, registros especiales, buses internos
 - ⇒ Unidad de control
- Arquitectura: Tipos de datos, repertorio, direccionamiento y secuenciamiento

Unidad de Memoria

- Almacena las instrucciones y los datos

Unidad de E/S

- Transfiere información entre el computador y los dispositivos periféricos

Buses

- Bus de datos: Para transferencia de datos entre la CPU y memoria o E/S
- Bus de direcciones: Para especificar la dirección de memoria o la dirección del registro de E/S
- Bus de control: Señales de control de la transferencia (reloj, lectura/escritura, etc.)

Partes de la CPU y sus funciones

- Registros
 - Visibles
 - Control/Estado
- ALU, FPU
- Control & Decode
 - Decodificación de instrucciones
 - Señales de control
- Buses Internos
- Circuitería

Arquitecturas

- * Von Neumann
- * Harvard

Estructura de la E/S

- * Separada
- * Mapeada en memoria

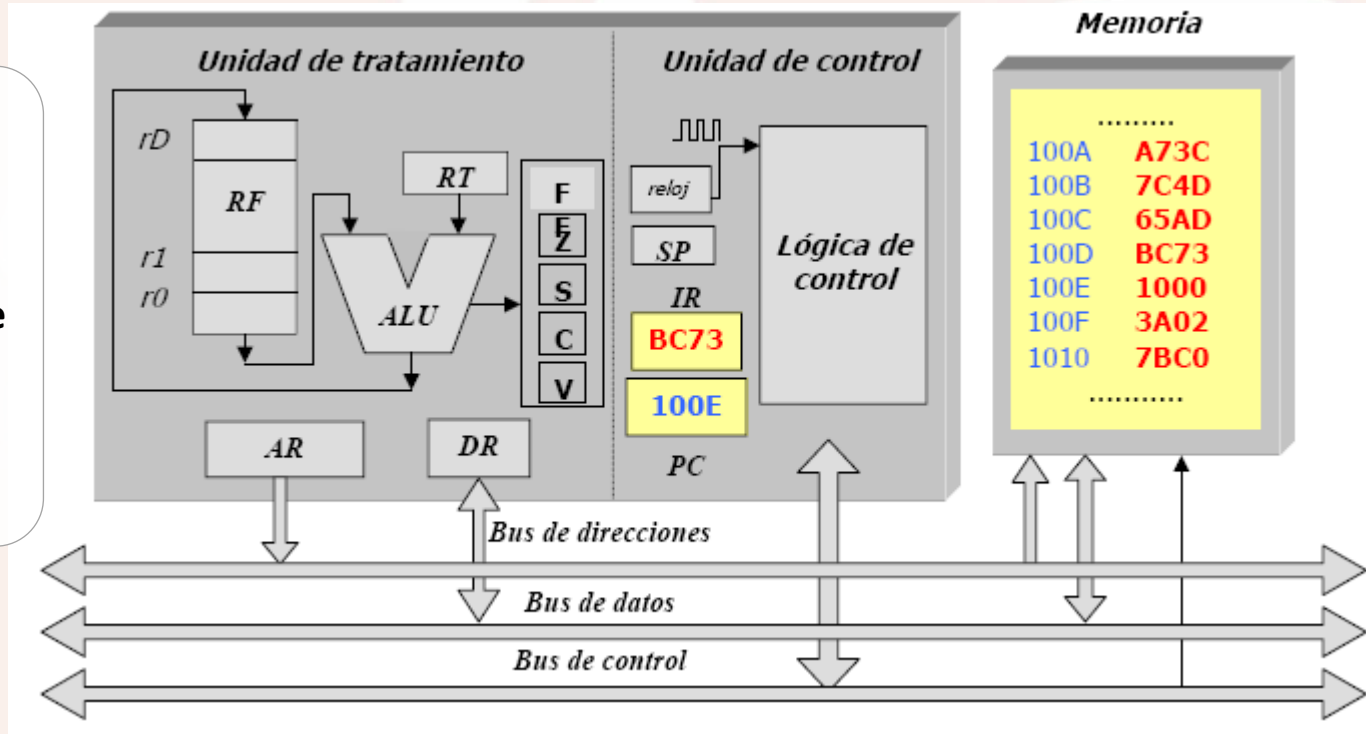
Ciclo de Búsqueda y Ejecución (fetch-execute cycle)

- BI: Las instrucciones se captan de memoria y se colocan en un registro especial (RI: registro de instrucción), para ser “interpretadas”
- EI: Ciertos bits en RI "controlan" las acciones siguientes:
 - Carga de operandos desde memoria
 - Ejecución de operaciones con los operandos
 - Almacenamiento del resultado en memoria
- Se calcula la dirección de la siguiente instrucción

Fase de búsqueda de la instrucción

Fase de Búsqueda de la instrucción

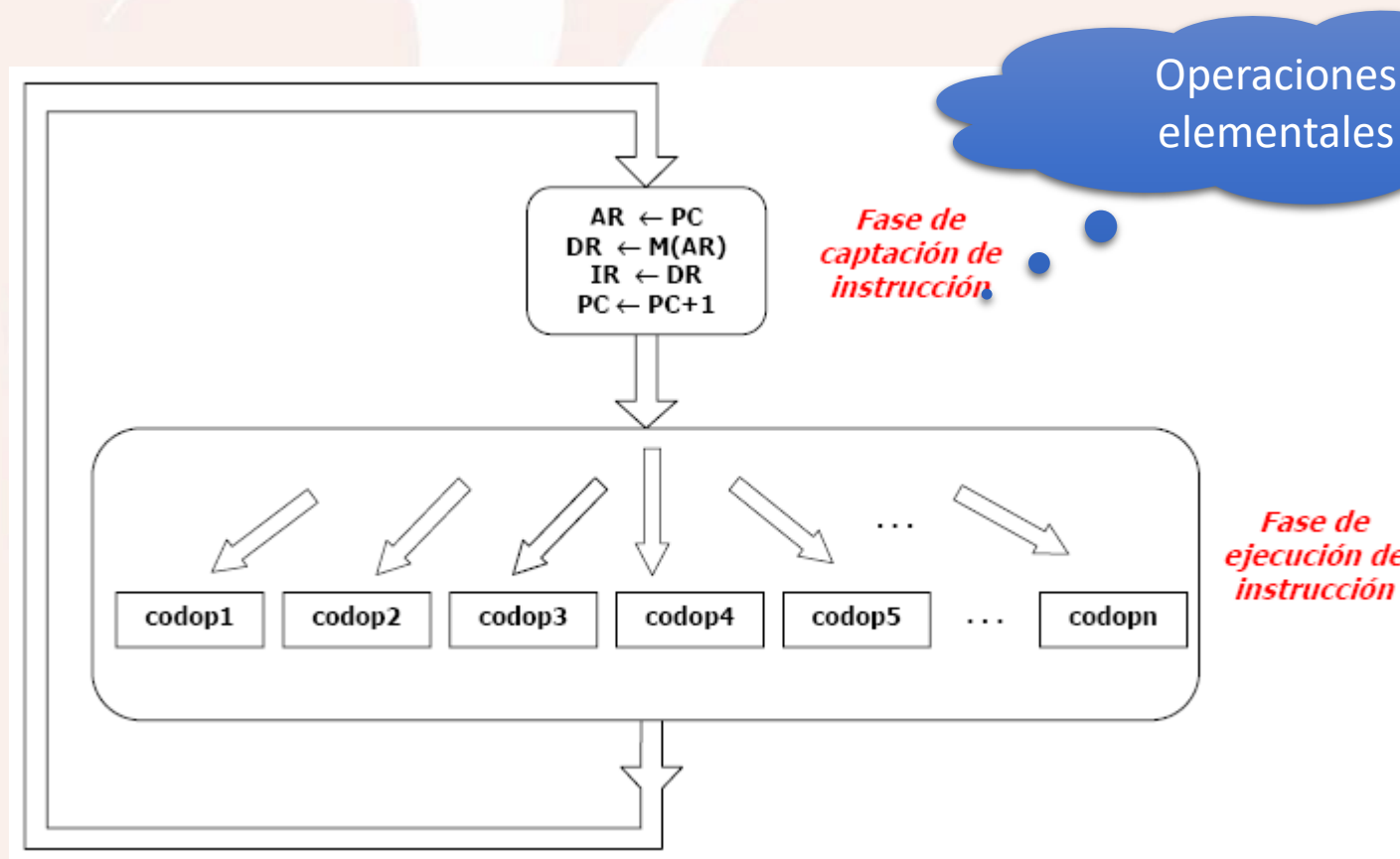
- $AR \leftarrow PC$
- $DR \leftarrow M(MAR)$: leer en memoria
- $IR \leftarrow DR$ (En IR queda almacenado el cod.op de la instrucción)
- $PC \leftarrow PC + 1$



Fase de ejecución

Fase de ejecución

- Operaciones específicas correspondientes al código de operación (**codop**) de la instrucción captada, generándose las señales de control oportuna



Ejemplo

- Indicar las operaciones elementales que se realizarán durante la ejecución de la instrucción, y los cambios que van teniendo lugar en PC, IR, AR, DR y r7.
- Computador de palabras de 16 bits.
- Instrucción:

LD r7, (rD)

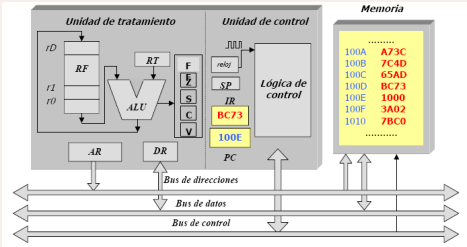
- $r7 \leftarrow M(rD)$
- $rD = 0x54C2$

Temporización: Instrucción de carga

Instrucción: LD r7, (rD)

Fase	Microoperacion	PC	IR	AR	DR	r7	rD
Fase Búsqueda de la instrucción (BI)							
Fase de dec							
Fase de ejecución de la instrucción							
BI							

direcciones	Contenidos	
0000	7AC4	
0007	65C9	
0039	0700	← instrucciones
003A	607D	
003B	2D07	
003C	C000	
54C2	D7A2	← dato
FFFF	3FC4	



Temporización: Instrucción de carga

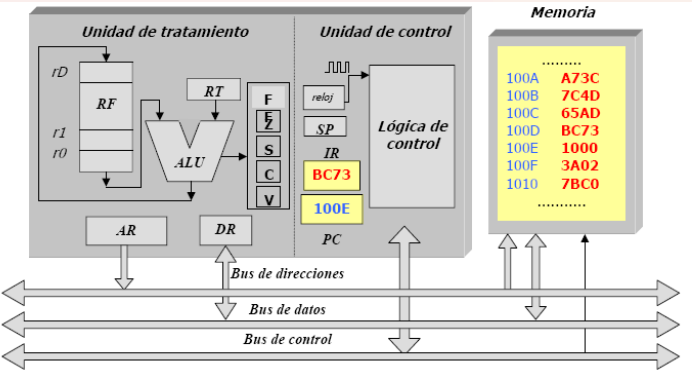
Instrucción: LD r7, (rD)

Fase	Microoperacion	PC	IR	AR	DR	r7	rD
		0039					54C2
Fase Búsqueda de la instrucción (BI)	AR←PC	0039		0039			54C2
	DR←M(AR), read	0039		0039	0700		54C2
	RI←DR	0039	0700	0039	0700		54C2
	PC←PC+1	003A	0700	0039	0700		54C2
Fase de decodific.	dec						
Fase de ejecución de la instrucción	AR←rD	003A	0700	54C2	0700		54C2
	DR←M(AR)	003A	0700	54C2	D7A2		54C2
	r7←DR	003A	0700	54C2	D7A2	D7A2	54C2
BI	AR←PC	003A	0700	003A			

¿Cuántos ciclos han sido necesarios para ejecutar la instrucción ?

¿Cuántos accesos a memoria?

direcciones	Contenidos	
0000	7AC4	
0007	65C9	
0039	0700	← instrucciones
003A	607D	
003B	2D07	
003C	C000	
54C2	D7A2	← dato
FFFF	3FC4	



Ejercicio

- Indicar las microoperaciones que se realizarán durante la ejecución de la instrucción, y los cambios que van teniendo lugar en PC, IR, DR, MBR y r7.

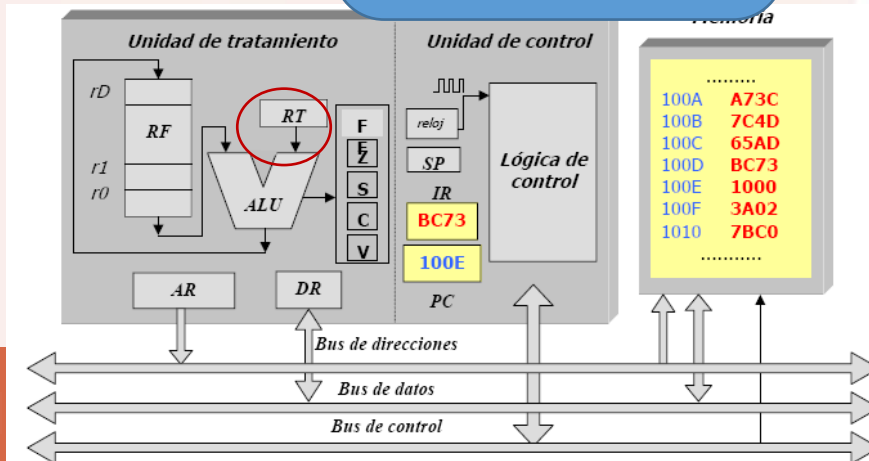
- Instrucción:

ADDS r1,r7,rD

- $r0 \leftarrow r7 + rD$
 - $rD = H'54C2$
- ¿Cómo quedan los indicadores C; S; Z; P; V?

Temporización: Instrucción de carga

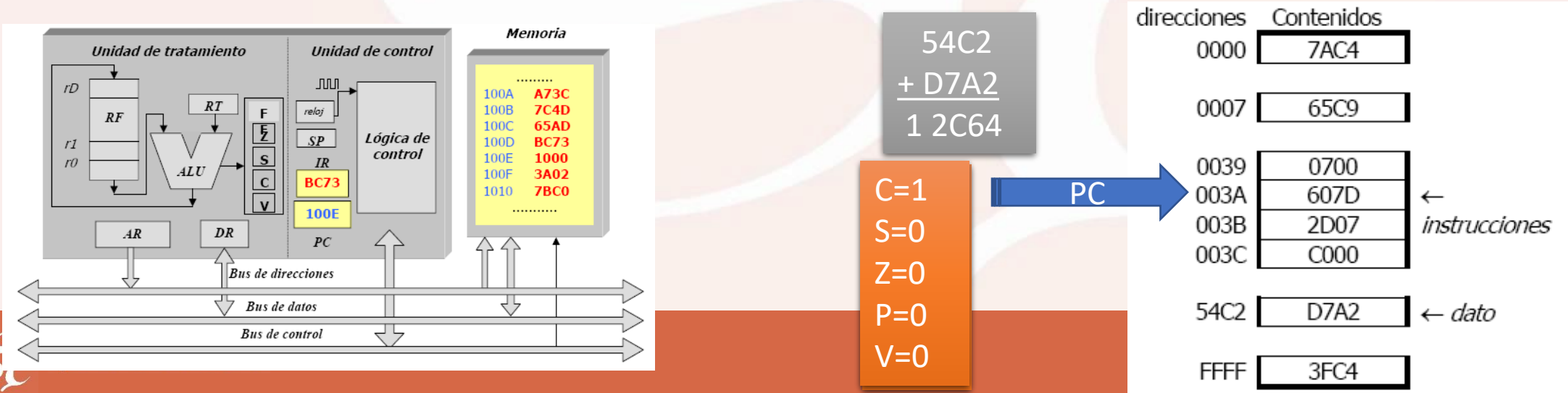
Fase	Microoperacion	PC	IR	AR	DR	r7	rD
		003A	0700	003A		D7A2	54C2
Fase Búsqueda de la instrucción (BI)	$AR \leftarrow PC$						
	$DR \leftarrow M(AR)$, read						
	$RI \leftarrow DR$						
	$PC \leftarrow PC + 1$						
Fase de ejecución de la instrucción							
BI							



direcciones	Contenidos	
0000	7AC4	
0007	65C9	
0039	0700	
003A	607D	← instrucciones
003B	2D07	
003C	C000	
54C2	D7A2	← dato
FFFF	3FC4	

Temporización: Instrucción de suma

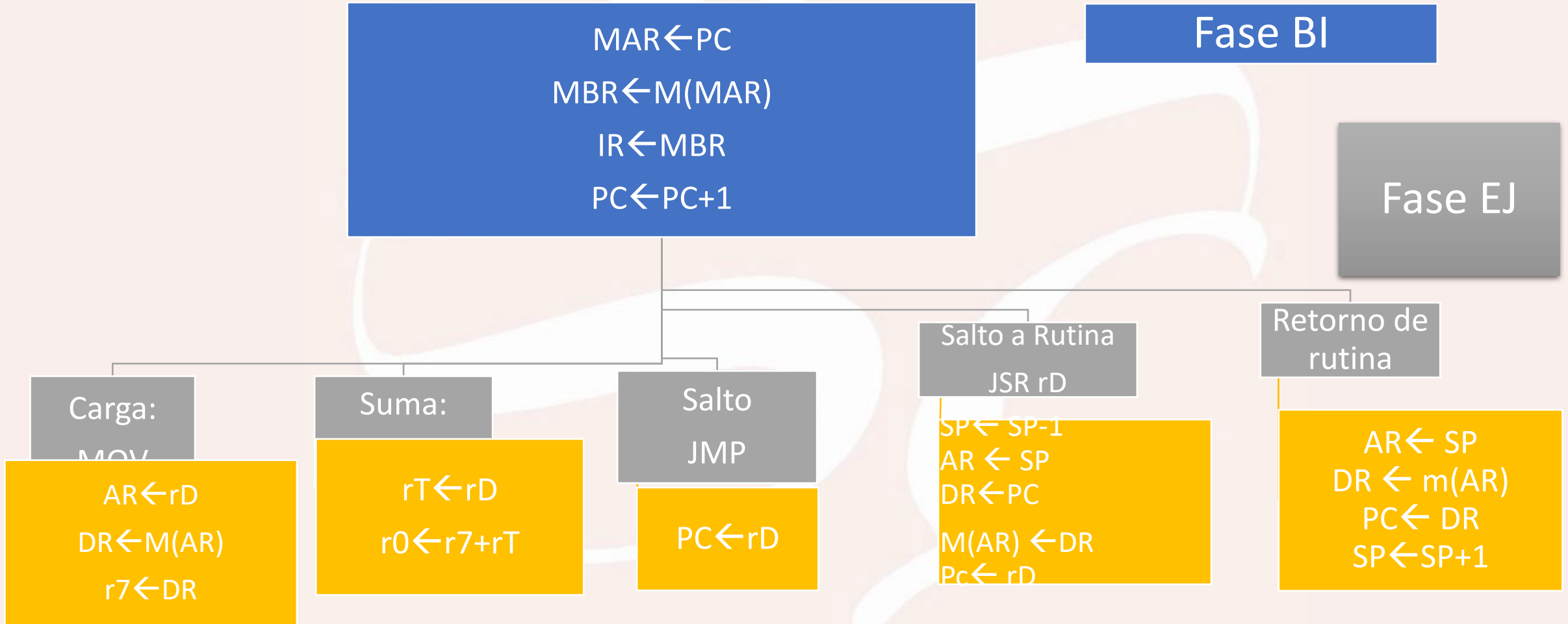
Fase	Microoperacion	PC	IR	MAR	MBR	r7	rD	r0
VALORES INIC.		003A	0700	54C2	D7A2	D7A2	54C2	
Fase Búsqueda de la instrucción (BI)	MAR←PC	003A	0700	003A	D7A2	D7A2	54C2	
Fase de ejec. de la instrucción								
BI								



Grupos de instrucciones

- Las instrucciones del lenguaje de programación se pueden clasificar:
 - **Instrucciones de transferencia de datos:** mover, cargar, almacenar, entrada, salida,...
 - **Instrucciones de tratamiento:** sumar, comparar, and, or, ...
 - **Instrucciones de flujo de control** o de bifurcación y saltos
 - Otras instrucciones

Temporización



Ejercicio

Si la memoria de instrucciones es de 32 bits, pero direccionable por byte, para actualizar el contador de programa (PC), para que apunte a la siguiente instrucción

¿En cuántos bytes de debe incrementar el contador de programa?

$$PC \leftarrow PC + 4$$

Requiere sumarle 4 ($PC \leftarrow PC + 4$), por ser una memoria direccionable por bytes y una arquitectura con tamaño de palabra de 32 bits

Arquitectura y factores que influyen en el rendimiento

- Rendimiento del procesador
 - Número real de instrucciones que se ejecutan (**N**) (No es necesariamente igual al número de instrucciones del programa objeto)
 - Número de ciclos que tarda cada instrucción: Ciclos por Instrucción (**CPI**) (número medio de ciclos de reloj del procesador para ejecutar una instrucción)
 - Tiempo de ciclo (**tc**) (es el tiempo que dura el ciclo de reloj del procesador → $1/F$: Frecuencia)

- Ecuación básica del rendimiento:

$$T = N \times CPI \times tc$$
$$T = N \times CPI / F$$

- Ecuación del rendimiento considerando los accesos a memoria
 - Accesos a memoria (**AMI**): es el número medio de accesos a memoria por instrucción
 - Tiempo de acceso a memoria (**ta**)

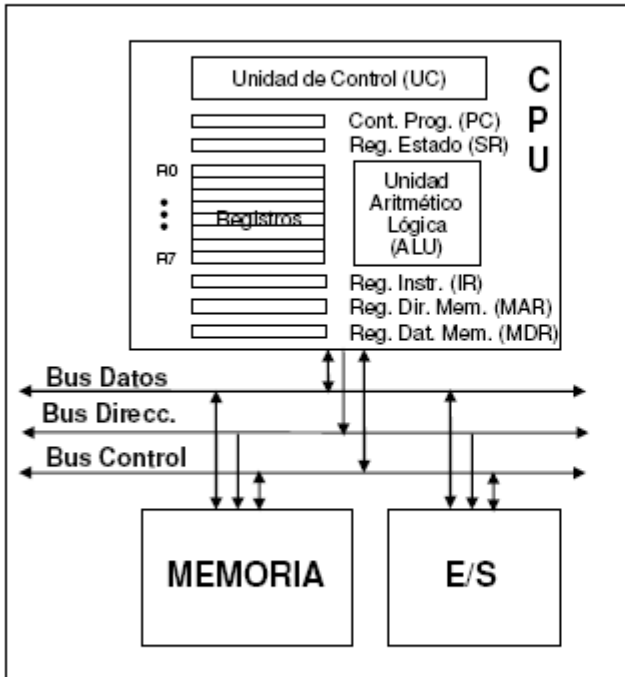
$$T = N \times CPI \times tc + N \times AMI \times ta$$

- **Objetivo:** reducir **T**: tiempo de ejecución

Resumen:

Estructura básica de un computador

Esquema



Módulos básicos

CPU (Unidad Central de Proceso)

- Realiza la ejecución de las instrucciones
- Bloques:
 - ⇒ Unidad de proceso (ruta de datos): ALU, banco de registros, registros especiales, buses internos
 - ⇒ Unidad de control
- Arquitectura: Tipos de datos, repertorio, direccionamiento y secuenciamiento

Unidad de Memoria

- Almacena las instrucciones y los datos

Unidad de E/S

- Transfiere información entre el computador y los dispositivos periféricos

Buses

- Bus de datos: Para transferencia de datos entre la CPU y memoria o E/S
- Bus de direcciones: Para especificar la dirección de memoria o la dirección del registro de E/S
- Bus de control: Señales de control de la transferencia (reloj, lectura/escritura, etc.)

Partes de la CPU y sus funciones

- Registros
 - Visibles
 - Control/Estado
- ALU, FPU
- Control & Decode
 - Decodificación de instrucciones
 - Señales de control
- Buses Internos
- Circuitería

Arquitecturas

- * Von Neumann
- * Harvard

Estructura de la E/S

- * Separada
- * Mapeada en memoria

Introducción a la Estructura y Arquitectura de Computadores

- Tema 1: Introducción a la Estructura y Arquitectura de Computadores